



惠州学院

HUIZHOU UNIVERSITY

毕 业 论 文 （设 计）

中文题目： 基于技术交流群体的智能聊天室

设计与实现

英文题目： Design and Implementation of an

Intelligent Chat Room for Technical

Communication Groups

姓 名 麦旻轩

学 号 2214100540

专业班级 22 软件工程 1 班

指导教师 刘利副教授

提交日期 2026 年 3 月

教务部制

诚信声明

本人郑重声明：所呈交的毕业论文（设计），是本人在指导老师的指导下，独立进行研究工作所取得的成果，成果不存在知识产权争议，除论文（设计）中已经注明引用的内容外，本论文（设计）不含任何其他个人或集体已经发表或撰写过的作品成果。对本论文（设计）的研究做出重要贡献的个人和集体均已在论文（设计）中以明确方式标明。本声明的法律结果由本人承担。

本人签名：

年 月 日

摘 要

面向技术讨论的在线交流，不只是发送普通文本消息，还经常伴随代码片段、报错日志、接口参数、文件附件和阶段性结论。通用聊天工具虽然能够完成基础沟通，但在历史回读、技术内容展示和讨论结果沉淀方面仍然不够贴合开发场景。针对这些问题，设计并实现了一套面向技术交流群体的智能聊天室系统 Mini-Chat-Bar。

系统采用前后端分离架构。前端使用 Vue 3 构建 Web 界面，并通过 Electron 封装桌面端；后端基于 Node.js、Express 和 Socket.IO 实现接口服务与实时通信；MongoDB 用于保存用户、房间、私聊消息、聊天室消息、收藏和 AI 相关数据。围绕技术交流场景，系统实现了注册登录、私聊、聊天室、文件分享、历史回读、收藏管理、AI 问答和讨论总结等功能。AI 模块采用检索增强生成思路，在聊天室问答时结合历史消息索引结果组织上下文，使返回内容尽量贴近当前讨论。在部署方式上，系统也保留了本地私有化部署思路，业务服务与数据库可按 Docker 容器方式部署在个人开发机或团队内网环境中，便于控制消息数据和业务配置的留存位置。

测试结果表明，系统已经跑通主要业务链路。登录、资料读取、消息列表和收藏列表等普通接口保持毫秒级响应；在单机本地环境下，100 个 Socket 客户端广播成功率保持 100%；AI 问答和讨论总结能够正常返回并写入历史记录，但耗时明显高于本地业务接口，主要受外部模型服务影响。整体来看，完成的系统已经能够满足课程项目和毕业设计演示需求。

关键词：智能聊天室；实时通信；AI 助手；Vue 3 框架；RAG 技术

Abstract

Technical-oriented online communication involves not only plain text messages but also code snippets, error logs, API parameters, file attachments, and interim conclusions. General-purpose chat tools can support basic communication, but they are less suitable for developer scenarios that require efficient history review, technical content display, and knowledge retention. To address this gap, this paper designs and implements Mini-Chat-Bar, an intelligent chat system for technical communities.

The system adopts a separated front-end and back-end architecture. The front end uses Vue 3 for web interfaces and Electron for desktop packaging, while the back end uses Node.js, Express, and Socket.IO for API services and real-time communication. MongoDB stores users, chat rooms, private messages, group messages, favorites, and AI-related data. For technical communication scenarios, the system supports user authentication, private messaging, group chat, file sharing, message history review, favorite management, AI-based question answering, and discussion summarization. The AI module uses retrieval-augmented generation to incorporate indexed historical messages into the current context. The system also supports local private deployment through Docker containers on personal development machines or internal networks, which helps control data storage and configuration.

Experimental results show that the system completes the main business workflows. Common interfaces such as login, profile retrieval, message lists, and favorites maintain millisecond-level response times. In single-machine tests, message broadcasting remained stable with 100 Socket clients. AI-based question answering and discussion summarization worked correctly and were recorded in message history, but their latency was significantly higher than local services because they depended on external model services. Overall, the system meets the needs of course projects and graduation project demonstrations.

Key words: Intelligent Chat Room; Real-time Communication; AI Assistant; Vue 3; RAG Technology

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 即时通讯系统与协作平台的发展	1
1.2.2 Web 实时通信技术研究现状	2
1.2.3 AI 辅助交流技术研究现状	2
1.3 研究内容与目标	2
1.4 论文结构安排	3
2 开发技术简介	5
2.1 技术选型与架构概述	5
2.2 前端实现技术	6
2.3 后端服务与实时通信技术	6
2.4 数据存储与消息组织技术	7
2.5 AI 辅助技术	7
3 系统需求分析	9
3.1 需求概述	9
3.2 业务需求分析	9
3.2.1 用户分析	9
3.2.2 业务场景描述	10
3.2.3 业务流程分析	11
3.3 功能需求分析	11
3.3.1 用户管理模块	12
3.3.2 即时通讯模块	12
3.3.3 聊天室模块	13
3.3.4 AI 助手智能问答模块	13
3.3.5 收藏模块	13
3.4 非功能需求分析	14
3.4.1 性能需求	14
3.4.2 安全性需求	14
3.4.3 可用性需求	14

3.4.4 数据分析需求	14
3.4.5 可扩展性与可维护性需求	15
3.5 用户用例分析	15
3.5.1 技术交流人群用例分析	15
3.5.2 个人开发者用例分析	16
3.5.3 普通交流用户用例分析	17
4 系统设计	19
4.1 系统总体设计	19
4.1.1 系统架构设计	19
4.1.2 系统总体流程设计	20
4.1.3 系统功能结构设计	21
4.2 功能模块设计	22
4.2.1 用户管理模块	22
4.2.2 即时通讯模块	23
4.2.3 聊天室模块	24
4.2.4 AI 助手智能问答模块	25
4.2.5 收藏模块	26
4.3 界面设计	27
4.3.1 登录与注册界面	27
4.3.2 主聊天业务界面	28
4.3.3 沉浸式技术聊天室界面	29
4.4 数据库设计	30
4.4.1 E-R 图设计	30
4.4.2 数据库逻辑结构设计	30
4.5 核心底层机制与部署拓扑设计	34
4.5.1 WebSocket 即时通信协作机理	34
4.5.2 系统物理部署与网络拓扑架构	35
4.5.3 系统核心数据流设计	36
5 系统实现	38
5.1 后端实现	38
5.1.1 核心模块实现	38
5.1.2 接口设计	45

5.1.3 示例数据	47
5.1.4 通信机制说明	48
5.1.5 接口联调展示	49
5.2 Web 前端实现	50
5.2.1 登录/注册页面	50
5.2.2 聊天主界面	52
5.2.3 AI 对话界面	54
5.2.4 消息列表与会话管理	55
5.2.5 设置与个人中心	57
5.2.6 Web 端消息流转与状态管理	59
5.2.7 容器化本地部署	59
5.3 桌面端实现	59
5.3.1 系统托盘功能	59
5.3.2 本地通知	60
5.3.3 与后端和 WebSocket 通信	61
6 系统测试	62
6.1 测试环境与测试方法	62
6.1.1 测试环境配置	62
6.1.2 测试方法与策略	63
6.2 功能测试	64
6.2.1 功能测试用例设计原则	64
6.2.2 接口与实时通信测试	64
6.2.3 Web 端功能测试	66
6.2.4 桌面端功能测试	70
6.2.5 功能测试结果总结	71
6.3 性能测试	72
6.3.1 响应时间测试	72
6.3.2 并发性能测试	73
6.3.3 资源占用测试	75
6.3.4 性能测试结果总结	77
7 总结与展望	78
7.1 工作总结	78

7.2 实践体会	78
7.3 系统不足与展望	79
参考文献	81
致谢	83

1 绪论

1.1 研究背景与意义

面向技术讨论的在线交流，与日常社交聊天并不一样。在课程项目联调、接口排错、版本确认和开源协作等场景中，参与者反复交换的往往是代码片段、报错日志、接口参数、测试结果和临时结论。消息一多，真正有用的内容就容易被后续讨论淹没，后来加入的人也难以及时接上上下文，只能重新追问，沟通成本随之增加。

现有聊天工具能够承担部分交流任务，但在技术讨论场景中仍有明显不足。通用社交软件偏向日常沟通，对技术内容展示、历史回看和结果留存考虑不多；功能更完整的协作平台覆盖面虽广，却往往带来更高的部署和使用成本。因而，围绕讨论过程组织消息、保留关键结论并支持后续回看，已成为技术交流系统的现实需求^[2]。如果系统还能支持个人电脑或团队内网中的本地部署，就能更好控制消息数据和讨论记录的留存位置。技术讨论中还常有一类高频需求：遇到陌生术语、报错信息或代码片段时，希望直接在当前界面完成划词解释，而不是反复切出聊天窗口搜索。

与此同时，大语言模型和检索增强技术的发展，也让聊天系统有了新的扩展方向。AI 的价值不在于脱离上下文单独回答问题，而在于结合当前会话历史继续参与问答，在需要时完成问题梳理、阶段总结和术语解释^[2]。这样一来，用户不必在聊天工具、搜索工具和独立 AI 工具之间来回切换，沟通链路更短，信息也更容易沉淀。因此，设计一套面向技术交流群体的智能聊天室系统，具有较强的实际应用价值和明确的研究意义。

1.2 国内外研究现状

1.2.1 即时通讯系统与协作平台的发展

现有即时通信系统的研究，早已不只停留在“发送一条消息”这一层面。消息留痕、群组组织、文件共享、权限控制和安全通信，已经逐渐成为这类系统的常见能力。针对具体使用场景对聊天系统做定制化裁剪，也是近年来较常见的实现思路。

这对本文的启发比较直接。技术交流群体需要的不是一套笼统的大平台，而是围绕问题讨论本身展开的轻量化工具：消息要清楚，历史要能回看，关键内容要能保留下来，后加入的人也要能尽快看懂前文。

1.2.2 Web 实时通信技术研究现状

在 Web 场景下，实时通信问题已经有比较成熟的处理方式。早期轮询和长轮询能够实现近实时效果，但请求频率较高，服务端压力也更明显。WebSocket 普及之后，浏览器与服务器可以保持持续连接，消息推送、在线状态同步和多人广播都更适合放在这一机制下完成^[7]。

从工程角度看，当前更常见的做法是把普通请求和实时连接分开使用：前者负责登录鉴权、数据读取和消息落库，后者负责在线状态、增量通知和广播同步。这样处理的好处很务实，链路更清楚，排查问题也更直接。本文的系统设计也是建立在这一思路之上。

1.2.3 AI 辅助交流技术研究现状

近年的 AI 研究已经从单纯追问“模型能不能回答问题”，逐渐转向“模型怎样结合外部信息更稳定地回答问题”。已有研究表明，大语言模型在问答、归纳和文本生成方面表现出较强能力，但当输入缺少上下文时，回答仍然容易变得空泛，甚至偏离原问题^[7]。

为了解决这一问题，检索增强生成成为近几年较受关注的方向。它的基本思路不是让模型只依赖参数记忆，而是在生成之前补充与当前问题相关的外部信息，从而提高回答的相关性和可用性^[7]。

针对开发者聊天场景中的阶段总结与多轮追问，已有研究也开始关注摘要方法、对话历史管理以及长上下文利用问题^[7]。

对技术交流场景来说，这一点尤其重要。同一句提问，放在不同聊天室、不同项目阶段里，含义可能完全不同。AI 如果不能接入历史讨论，就很难真正理解当前问题。也正因为如此，本文把 AI 看成聊天链路的一部分，而不是一个脱离场景的独立问答入口。

1.3 研究内容与目标

本文以本人设计并实现的 Mini-Chat-Bar 为对象，围绕技术交流群体的使用特点，完成一套轻量化智能聊天室系统的分析、设计、实现与测试。研究内容主要包括四个方面：一是结合课程项目协作、联调排错和日常技术讨论等场景，梳理目标用户、业务流程与核心需求；二是完成系统总体架构、功能模块和数据组织设计，形成支撑实时通信与 AI 辅助的技术方案；三是实现私聊、聊天室、历史回读、收藏、侧边栏 AI 对话、结合聊天记录的上下文问答、讨论总结与划词

解释等功能；四是通过功能测试和性能测试验证系统的可用性与稳定性。

本文的目标不是泛泛讨论“智能聊天”这一概念，而是做出一套能够实际运行的产品原型 Mini-Chat-Bar，并验证它是否适合技术交流场景。该产品既要满足消息收发、历史回看和多人协作等基础交流需要，也要让 AI 以更自然的方式融入讨论过程，例如在侧边栏继续追问、在聊天室里结合历史消息回答问题、对选中的文本做解释，并把有价值的结果保留下来，减少重复沟通。

从价值上看，这套系统可以为课程项目、小型开发团队和技术兴趣社群提供一套门槛相对较低的交流工具，帮助用户更快定位问题、理解术语并回看已有结论。从实现特点上看，本文的侧重点不在于提出新的模型算法，而在于把即时通信、历史消息组织和 AI 辅助放到同一产品链路里：AI 不是独立悬浮在外的问答窗口，而是能够利用聊天记录参与当前讨论，回答结果又能继续被引用、收藏和回看。这种围绕具体使用场景进行整合的做法，也是本课题相对有新意、同时更有现实意义的地方。

1.4 论文结构安排

后文按“需求分析、总体设计、具体实现、测试验证”的顺序展开。

第一章为绪论，交代课题背景，并梳理即时通信、实时通信与 AI 辅助相关的研究现状。

第二章介绍相关技术，说明前端界面组织、后端服务处理、实时通信、数据存储及 AI 检索增强等技术在本项目中的使用方式。

第三章进行系统需求分析，围绕目标用户、典型场景、功能需求与非功能需求，明确系统需要完成和暂不实现的内容。

第四章阐述系统总体设计，包括整体架构、模块划分、数据库设计，以及认证、消息流转、聊天室交互和 AI 辅助等核心流程。

第五章说明系统实现，结合前后端代码与界面，介绍用户、聊天、聊天室、收藏、AI 模块及 Electron 桌面端功能的具体实现。

第六章给出系统测试，从测试环境、功能测试和性能测试三个方面验证运行情况，展示接口、实时通信和 AI 调用链路的测试结果。

第七章为总结与展望，归纳已完成的工作，并说明后续仍可继续改进的方向。

2 开发技术简介

2.1 技术选型与架构概述

本章不再把项目中用到的技术逐个写成教材式介绍，而是围绕 Mini-Chat-Bar 的实际需求说明技术选型的理由。开发过程中，系统最先暴露出的并不是某个单独页面的问题，而是几类能力需要同时成立：既要完成登录、资料读取、消息保存和收藏回看等稳定的数据处理，也要支撑私聊提醒、聊天室广播、在线状态变化和 AI 增量回复这类实时交互，还要兼顾桌面端通知与托盘常驻等本地能力。单靠一种技术很难把这些事情都处理好，因此项目最终采用了前后端分层、实时通信与数据存储协同配合的方案。

如图 2-1 所示，系统整体可以分为接入层、前端应用层、服务层以及数据与智能辅助层。这样的划分不是为了把架构图画得复杂，而是因为系统中的不同任务确实需要不同链路来承担。前端主要负责界面呈现、会话切换和用户交互；服务层负责权限校验、业务处理和消息流转；数据层负责用户资料、聊天记录和收藏内容的持久化；AI 服务层则负责问答、总结和解释等增强能力。这样拆开以后，稳定的数据读写与需要即时反馈的实时交互各走合适的路径，系统结构会更清楚，后续实现和排查问题也更直接。

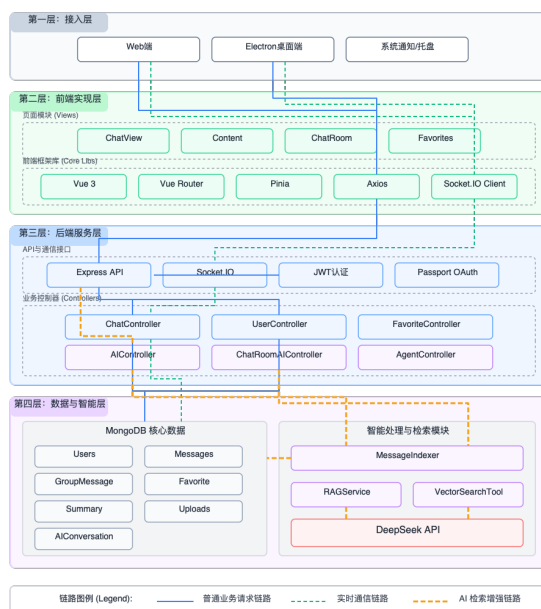


图 2.1 系统技术架构图

图 2-1 并不是套用通用模板得到的，而是依据本系统已经完成的功能结构整

理出来的。图中把用户管理、即时通信、聊天记录、文件管理和 AI 辅助几个部分放在同一套架构下，说明它们如何共同支撑私聊、聊天室、收藏回看和智能问答等功能。这样的表达方式也为后续章节展开总体设计、功能实现与测试验证提供了统一的基础。

2.2 前端实现技术

前端部分最明显的特点，是同一套界面中同时存在登录鉴权、会话切换、实时消息显示、聊天室详情、收藏回看和 AI 辅助等多种交互。Vue 3 在这个项目里的价值，主要体现在把这些界面拆成职责相对清楚的部分，而不是单纯为了追求新框架^[2]。聊天主界面采用会话列表、消息区和详情区的三栏结构后，私聊、聊天室和收藏等功能就可以围绕各自的区域展开，界面关系也更容易理清。

状态管理由 Pinia 承担，但并没有把所有界面状态都收进全局。跨页面共享的会话状态和消息数据由状态管理统一维护，局部弹窗、搜索词和短时交互则继续放在组件内部处理。这样的划分更贴合本项目的实际需要，也减少了页面联动时的复杂度^[2]。配合 Vue Router 管理页面切换、Vite 支撑开发阶段的快速调试，再结合 Electron 补足托盘、通知和桌面常驻能力，前端部分既能保持 Web 端开发效率，也能满足桌面端展示需求。

Electron 的接入同样是围绕现有页面补能力，而不是另写一套桌面客户端。这样做保留了 Web 界面的复用效率，同时也补上了桌面常驻、消息提醒和窗口切换这些在实际使用和答辩演示中都比较直观的交互能力。

2.3 后端服务与实时通信技术

后端采用 Node.js 和 Express，并不是因为它们在 Web 项目里足够常见，而是因为从这个系统从一开始就同时存在两类请求形态：一类是登录、资料读取、消息查询和收藏处理这类可以较快结束的普通请求，另一类是消息发送、在线同步和 AI 流式回复这类会持续交互的链路。把这两类请求放在同一套 JavaScript 运行环境里处理，有利于减少前后端联调时的切换成本^[2]。

本项目里 HTTP 和 Socket.IO 的分工很清楚，二者并不是互相替代。普通请求主要负责身份认证、数据读写和结果持久化，实时通道主要负责把新消息、在线状态和增量内容及时同步给在线成员。这样处理的好处很实际：稳定数据通过普通接口处理，实时变化通过长连接同步，既便于保证消息可靠落库，也便于后续定位问题^[2]。

身份认证方面，系统采用 JWT 维护登录状态。无论用户采用邮箱密码、验证码还是第三方登录，后续访问都统一进入同一套鉴权流程。这样处理既方便权限控制，也减少了不同登录方式带来的后端分支复杂度^[2]。

2.4 数据存储与消息组织技术

从数据组织看，虽然私聊消息、聊天室消息、收藏快照、AI 回复和讨论总结最终都会显示在聊天界面里，但它们的结构、读取方式和保存重点并不相同。私聊更强调双方关系下的历史回读，聊天室更强调按房间分页加载和多人共享，收藏更强调快照留存，AI 对话和总结则需要保留上下文信息。基于这种差异，系统采用 MongoDB 存放主要业务数据^[2]。

结合实际访问路径，系统把用户、聊天室、消息、收藏和 AI 相关数据分别组织，历史消息重点围绕会话或房间读取，收藏内容则便于后续筛选和回看。文件资源与消息记录分开保存，只在消息中保留必要的展示与定位信息。这样的设计不追求形式上的复杂，而是服务于当前项目里最常见的读取与回看需求。

索引设置同样跟着读取方式走。常用查询集中在最近会话、按时间回读历史消息、按房间查看讨论和按用户查看收藏等场景，因此数据库优化也围绕这些高频路径展开。对本科毕业设计的系统规模来说，这样的组织方式已经足够支撑消息分页、历史检索和收藏沉淀等主要功能。

2.5 AI 辅助技术

本系统中的 AI 不是附加在聊天界面旁边的演示功能，而是直接参与技术讨论过程。侧边栏 AI 助手更适合单人连续追问，也可以用于对选中的术语、报错信息或代码片段做解释；聊天室内的 AI 问答和阶段总结则面向多人协作，回答结果能够继续留在房间里被所有成员查看和引用。

如果只把用户最后输入的一句话直接交给模型，回答很容易脱离前文。为了解决这个问题，项目在聊天室问答链路中加入了消息索引与检索增强。问答发起前，系统会先从当前房间的历史消息中召回相关讨论，再把这些上下文与用户问题一并提交给模型。这样得到的回答更贴近房间里已经形成的讨论脉络，而不是把每次提问都当作一条孤立的新问题。

考虑到 AI 回复明显慢于普通接口，系统采用流式输出方式逐步返回内容，让提问者和房间成员都能及时看到回答进展。这种做法虽然没有改变模型本身的响应速度，但能明显改善交互体验。

当前实现还保留了一个务实的降级方案：当外部向量或 embedding 服务不可用时，系统仍可退回本地检索方式，先保证开发环境和答辩环境能够跑通整条链路。结合历史消息回读、聊天室总结与 AI 回复持久化功能，AI 在本系统里的角色更接近“利用已有讨论提供辅助”，而不是替代用户完成全部交流。

为了更清楚地说明大语言模型与系统内业务数据的结合方式，图 2-2 给出了本系统 RAG（检索增强生成）的核心流程。

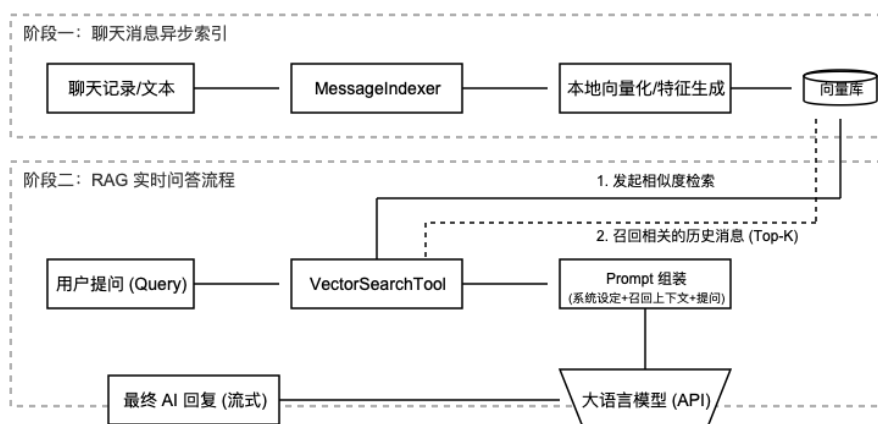


图 2.2 RAG 检索增强生成原理图

如图 2-2 所示，整个智能问答可以概括为“先组织历史上下文，再生成回答”两个环节。用户发送问题后，系统先整理与当前话题相关的历史消息，再将其与当前提问一并提交给模型；模型返回结果后，系统按普通消息或流式消息的形式展示出来。这套设计的重点不在于堆叠更多 AI 术语，而在于让回答尽量贴近当前聊天室已经形成的讨论上下文。

3 系统需求分析

3.1 需求概述

本章围绕 Mini-Chat-Bar 面向的实际使用场景展开需求分析。本文关注的不是一般社交聊天，而是技术讨论中的高频操作，例如登录后进入私聊或聊天室、发送代码和文件、回看历史消息、对关键内容做收藏，以及在无人及时回复时调用 AI 继续处理问题。第 1 章已经说明，这类场景对上下文连续性、消息组织方式和讨论结果沉淀都有更高要求，因此需要把需求拆得更细，不能只停留在“实现聊天功能”这一层。

结合当前系统定位，需求分析主要回答四个问题：系统服务哪些用户；这些用户在什么场景下使用系统；系统至少需要提供哪些功能；为了保证后续实现和演示，系统在性能、安全和可维护性方面应满足哪些基本要求。后文将按照业务需求、功能需求、非功能需求和用户用例四个部分展开。

3.2 业务需求分析

业务需求分析的目标是先把“谁在用、怎么用、为什么用”说清楚。只有用户、场景和流程明确，后续的模块设计和实现细节才有依据。

3.2.1 用户分析

根据本系统的使用目标，用户可以概括为技术交流人群、个人开发者和普通交流用户三类。三类用户都需要基础聊天能力，但关注重点并不相同。

（1）技术交流人群

技术交流人群包括课程项目组成员、实验室小团队、开源协作成员等。这类用户最常见的操作是在会话中发送代码片段、报错日志、接口参数和测试结果，然后围绕同一个问题持续讨论。他们关注的不是“聊得热闹”，而是问题能不能尽快定位、结论能不能保存、后来的人能不能看懂前面的讨论。因此，这类用户对聊天室、历史回读、引用回复、文件共享和 AI 辅助问答的依赖最强，这类面向问题定位的持续讨论方式，与面向任务协作的即时交流系统研究结论较为一致^[2]，使用流程如图 ?? 所示。

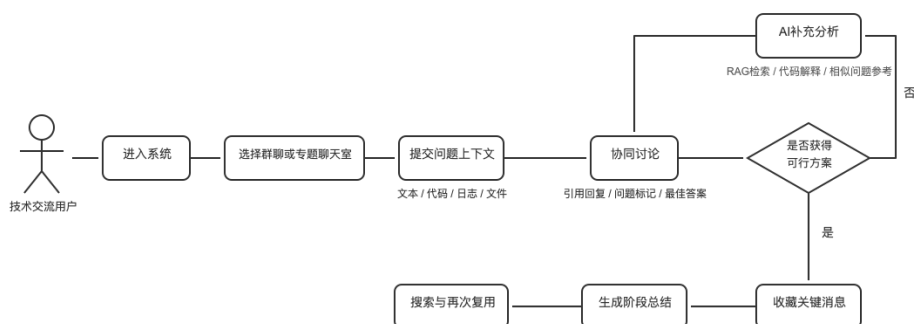


图 3.1 技术交流人群使用流程示意图

（2）个人开发者

个人开发者通常独立完成学习或开发任务，遇到问题时更倾向于先自己查资料、再向系统提问。他们对侧边栏 AI 助手、历史检索和收藏功能的需求更明显：一方面，希望快速得到代码解释、报错分析或实现建议；另一方面，希望把有价值的回答和讨论沉淀下来，形成个人资料库。当单人问答不足以解决问题时，这类用户也会临时创建聊天室，与其他开发者集中讨论。

（3）普通交流用户

普通交流用户可以是测试人员、产品人员或只需要跟进阶段进展的参与者。这类用户并不一定频繁发起技术讨论，但需要较低门槛地进入房间、查看阶段性结论、下载共享文件和回读历史消息。他们更看重界面是否直观、总结是否清楚，以及能否在不参与完整讨论过程的情况下快速了解当前进展。

3.2.2 业务场景描述

结合上述用户特征，可以归纳出本系统中较为典型的三类业务场景。

（1）场景一：多人协同定位接口异常

项目成员在聊天室中集中处理接口异常问题。发起者先发送报错信息、请求参数和相关代码，其他成员在同一房间继续补充判断；如果短时间内没有形成明确结论，可以直接在房间中触发 @AI 进行辅助分析。问题解决后，关键回答和结论还应允许被收藏，方便后续再次查阅。

（2）场景二：个人开发者快速验证实实现思路

个人开发者在实现某个功能时，可能先通过 AI 面板询问框架用法、错误原因或代码修改建议。如果单轮问答不能解决问题，用户可以新建一个与当前主题相关的聊天室，把问题发给其他成员继续讨论。讨论结束后，重要结论可以保存

在收藏页中，避免下次遇到同类问题时重复检索。

(3) 场景三：普通交流用户整理阶段性讨论结论

测试人员或产品人员未必全程参与讨论，但在查看功能变更、确认交付内容或准备汇报材料时，需要快速掌握前面的讨论结果。此时他们更依赖聊天总结、历史回读和共享文件下载等功能，以减少重新沟通的成本。

3.2.3 业务流程分析

从系统角度看，一次完整的业务流程通常包括登录进入、选择会话、发送或接收消息、触发 AI 辅助、整理关键信息和后续回看几个阶段，如图 ?? 所示。

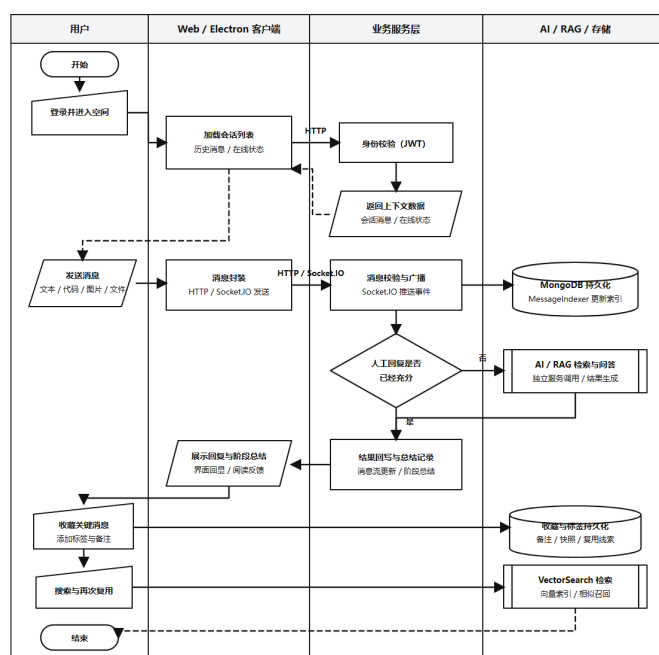


图 3.2 系统业务流程图

用户首先完成注册或登录，进入聊天主界面后选择私聊或聊天室。在交流过程中，系统需要支持文本、代码、图片和文件等多种消息形式；如果讨论涉及持续协作，还需要显示房间成员和在线状态。遇到技术问题时，用户可以在侧边栏 AI 面板单独提问，也可以在聊天室中直接发起 @AI 提问。讨论结束后，重要消息可以加入收藏，聊天室消息和 AI 回复应允许通过历史记录再次读取；对于设置了有效期的聊天室，任务结束后还应转为归档状态，供后续查阅。

3.3 功能需求分析

根据业务需求，系统功能可归纳为用户管理、即时通讯、聊天室、AI 助手和收藏五个核心模块，具体功能汇总见表 ??。各模块之间的结构关系将在第 4 章

系统设计部分结合功能结构图进一步展开说明。

表 3.1 系统功能需求汇总

功能模块	核心功能点	功能说明	主要服务对象
用户管理模块	注册登录、资料维护、在线状态	完成身份认证、用户资料读取与更新，并为后续会话和权限控制提供基础数据	全部用户
即时通讯模块	私聊、消息发送、历史回读、富媒体支持	承担私聊消息收发与历史回看，支持文本、代码、图片和文件等消息类型	全部用户
聊天室模块	房间创建、加入方式、成员状态、归档管理	提供面向技术讨论的多人协作空间，支持公开或密码加入、在线人数同步和到期归档	技术交流人群、个人开发者
AI助手模块	单人问答、房间问答、总结生成、上下文检索	在侧边栏或聊天室内提供技术问答、代码解释和讨论总结能力，并结合历史消息提升回答相关性	技术交流人群、个人开发者
收藏模块	收藏、分类、检索、回看来源	将关键消息、代码或结论保存下来，支持后续检索与再次回看	全部用户

3.3.1 用户管理模块

用户管理模块负责系统的身份入口和资料基础。它不仅要处理注册与登录，还要为会话页、聊天室和收藏页提供稳定的用户身份数据。

- (1) 支持邮箱密码登录、邮箱验证码登录以及第三方登录，保证不同用户可以顺利进入系统。
- (2) 登录成功后生成有效会话信息，并在页面刷新或重新进入时保持登录状态。
- (3) 支持读取和修改昵称、头像等基础资料，使聊天列表、消息记录和个人中心的信息保持一致。
- (4) 支持在线状态同步，为联系人列表和聊天室成员列表提供状态依据。

3.3.2 即时通讯模块

即时通讯模块是系统的核心链路，主要负责私聊消息的发送、接收、展示和历史回读。由于本系统面向技术交流，消息类型不能只限于普通文本。

- (1) 支持用户之间进行私聊，并能够及时同步新消息。
- (2) 支持文本、代码片段、图片、文件等多种消息形式，保证技术讨论内容能

够被完整表达。

- (3) 支持引用回复、消息状态反馈和历史消息分页读取，便于围绕具体内容继续讨论。
- (4) 支持最近会话、未读消息和消息搜索等基础能力，降低回看和定位信息的成本。

3.3.3 聊天室模块

聊天室模块主要服务于多人协作讨论，适合处理某个专题、某次发布或某个待解决问题。相比私聊，聊天室更强调成员组织、消息广播和阶段性归档。

- (1) 支持创建聊天室，并设置名称、技术方向、加入方式和有效期等基础信息。
- (2) 支持公开加入和密码加入等方式，满足不同讨论范围的需要。
- (3) 支持成员进入房间后的在线人数同步、历史消息回读和新消息广播。
- (4) 支持聊天室到期后转为归档状态，避免长期闲置房间继续产生无效消息。

3.3.4 AI 助手智能问答模块

AI 助手模块负责补充人工讨论中的空档期，帮助用户更快获得初步答案。该模块既要支持单人连续追问，也要支持结合聊天记录的多人的上下文问答，并承担术语解释与阶段总结等辅助任务，这与对话历史管理型问答系统的设计重点相吻合^[2]。

- (1) 支持侧边栏 AI 助手，供用户单独输入问题、代码或报错信息，并对选中的术语、文本或代码片段进行解释。
- (2) 支持在聊天室中通过 @AI 触发问答，使房间成员可以共同查看 AI 返回结果。
- (3) 支持结合历史消息进行检索增强，使 AI 回答尽量贴合当前讨论上下文，减少重复说明前情的成本。
- (4) 支持根据一段时间内的聊天内容生成讨论总结，便于后来加入的成员快速了解前情。

3.3.5 收藏模块

收藏模块用于把聊天过程中的临时信息转化为可回看的长期资料，这也是本系统区别于普通聊天工具的重要部分。

- (1) 支持对私聊消息、聊天室消息和 AI 回复执行收藏操作。
- (2) 支持为收藏内容增加备注或标签，方便后续按主题整理。

- (3) 支持在收藏页中查看已保存内容，并保留来源信息，便于回到原始会话继续阅读。
- (4) 支持基础检索与筛选能力，帮助用户快速定位历史问题和解决方案。

3.4 非功能需求分析

除业务功能外，系统还需要满足基本的性能、安全、可用性和可维护性要求。这些要求虽然不直接对应某个按钮或页面，但会直接影响系统能否稳定运行。

3.4.1 性能需求

本系统主要面向课程项目、小型技术团队和答辩演示场景，因此性能要求以“稳定可用”为主，而不是追求大规模平台级指标。普通业务接口如登录、资料查询、消息列表读取和收藏列表读取应保持较快响应，常见查询在教学演示规模下应控制在秒级以内；私聊和聊天室消息发送后需要及时回显，在线用户应在可感知实时范围内收到更新。对于 AI 功能，由于依赖外部模型服务，其耗时可以明显高于普通接口，但界面需要提供思考状态或流式反馈，避免用户误以为系统失去响应^[7]。

3.4.2 安全性需求

系统需要保证身份认证、访问控制和数据传输的基本安全。用户密码应采用单向散列方式保存，避免明文存储；登录后所有受保护接口都应进行身份校验，并限制未授权用户访问他人资料、私聊记录和受限聊天室。对于聊天室加入、文件上传和 AI 上下文读取，也需要结合当前用户权限做边界检查，防止越权读取或恶意输入影响系统稳定性^[7]。

3.4.3 可用性需求

系统界面应保持清晰直接，避免把高频操作埋得过深。登录、切换会话、发送消息、查看历史、触发 AI 和收藏内容都应在较少步骤内完成。对于技术交流场景来说，代码、文件和普通文本消息需要有明显区分，避免阅读混乱；对于桌面端和 Web 端，还应保证基础操作逻辑尽量一致，减少用户在不同端之间切换时的学习成本。

3.4.4 数据分析需求

虽然本系统不是数据分析平台，但为了后续优化系统运行和用户体验，仍需要保留一些基础统计项。这些数据主要服务于后续观察系统负载、房间使用

情况、AI 调用频率和收藏使用情况，不直接面向普通用户展示，核心数据项见表 ??。

表 3.2 系统数据分析核心数据项

数据类别	关键数据项	采集目的	更新方式
系统活跃数据	在线用户数、连接持续时间	观察系统负载和在线波动情况	实时记录/按日汇总
聊天室使用数据	房间创建数、过期房间数、成员活跃度	判断聊天室功能的实际使用情况	实时记录/按周汇总
消息流量数据	私聊消息量、聊天室消息量、文件消息量	估算消息存储和网络传输压力	实时记录/按日汇总
AI 调用数据	提问次数、总结次数、响应耗时	评估 AI 模块使用频率和调用开销	调用时记录/按日汇总
收藏沉淀数据	收藏总量、标签使用情况、检索频率	判断知识沉淀功能的利用效果	实时记录/按周汇总

3.4.5 可扩展性与可维护性需求

系统应保持清晰的模块边界，便于后续增加新消息类型、替换 AI 服务或扩展桌面端功能。前后端分离结构、接口分层和独立的控制器/路由设计有助于降低修改时的耦合度；同时，系统还应保留必要的日志与错误信息，便于联调、测试和后续维护。

3.5 用户用例分析

在需求分析阶段，用例的作用是从用户视角检验系统是否覆盖了主要操作路径。结合前文的用户分类，下面分别说明三类用户的典型用例。考虑到三类用户在本系统中的高频操作存在明显共性，本文不再把所有可能场景逐项铺开，而是为每类用户各选取两个最具代表性的用例：一个突出核心使用入口，另一个补充结果沉淀、回看或扩展协作路径。这样既能覆盖主要业务链路，也能避免用例描述在不同用户之间反复重复。

3.5.1 技术交流人群用例分析

技术交流人群的使用路径最完整，通常会同时涉及聊天室、消息回读、AI 提问和收藏等多个功能，用例如图 ?? 所示。

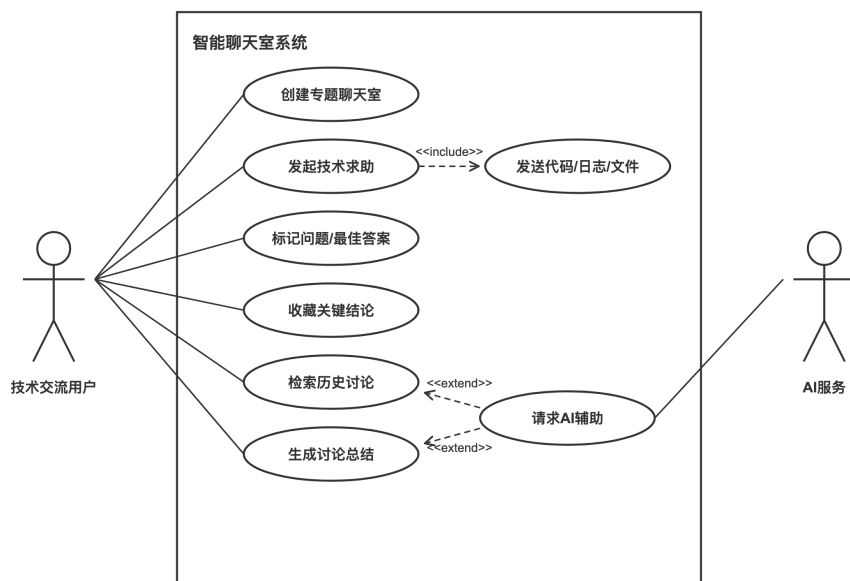


图 3.3 技术交流人群用例图

(1) 用例一：发起技术问题求助

当项目成员在开发或联调过程中遇到问题时，可按以下流程完成求助：

- ① 进入对应聊天室，发送问题描述、报错日志和相关代码片段。
- ② 其他成员在房间内继续补充分析，必要时通过 @AI 获取辅助回答。
- ③ 问题解决后，将关键消息或结论加入收藏，便于后续回看。

(2) 用例二：创建专题聊天室组织集中讨论

当某个任务需要多人集中协作时，可按以下流程操作：

- ① 创建聊天室，设置房间名称、技术方向、加入方式和有效期。
- ② 邀请相关成员加入房间，围绕同一主题集中讨论并共享文件。
- ③ 任务结束后保留历史记录，聊天室在到期后转为归档状态。

3.5.2 个人开发者用例分析

个人开发者的典型操作更偏向“先自己验证，再决定是否扩大讨论范围”，用例如图 ?? 所示。

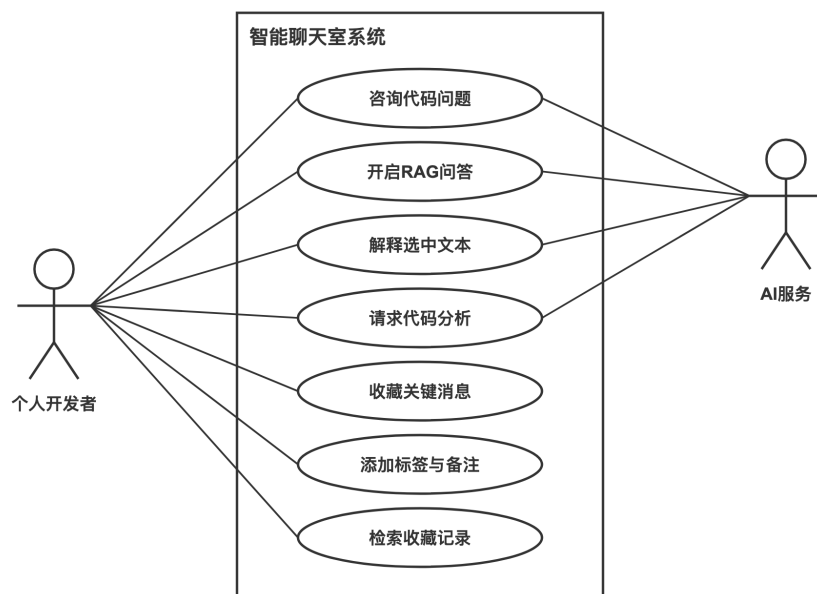


图 3.4 个人开发者用例图

(1) 用例一：向 AI 助手咨询代码问题

当用户独立开发时，可以通过以下流程获取辅助支持：

- ① 在 AI 面板中输入代码片段、报错信息或具体问题；遇到陌生术语或局部文本时，也可以直接触发解释。
- ② 系统返回解释、修改建议或进一步排查方向，连续追问时保留当前 AI 会话上下文。
- ③ 将有参考价值的回答保存到收藏页，方便今后再次查看。

(2) 用例二：收藏并整理关键讨论内容

当用户遇到有长期参考价值的内容时，可按以下流程处理：

- ① 对目标消息执行收藏操作，并补充必要的备注或标签。
- ② 后续在收藏页中按关键词或分类回看对应内容，形成个人资料积累。

3.5.3 普通交流用户用例分析

普通交流用户的操作相对简单，重点在于快速了解讨论进展和获取相关资料，用例如图 ?? 所示。

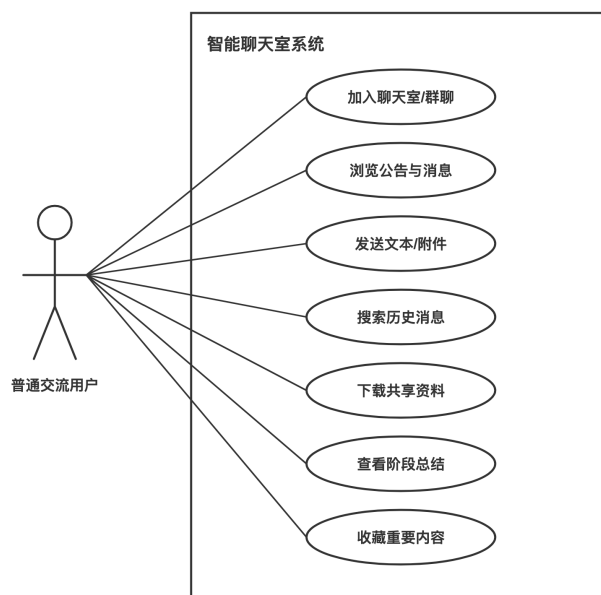


图 3.5 普通交流用户用例图

(1) 用例一：加入聊天室参与日常沟通

普通交流用户参与房间讨论时，可按以下流程进行：

- ① 通过公开入口或密码进入目标聊天室，查看公告和历史消息。
- ② 根据需要补充问题、确认进度或下载房间中共享的资料文件。

(2) 用例二：检索历史消息并获取阶段性结论

当用户需要快速了解某个问题的处理结果时，可按以下流程操作：

- ① 通过历史消息检索或聊天总结功能定位相关讨论内容。
- ② 查看总结结果、打开原始消息或下载关联文件，完成信息回读。

本章从用户、场景、流程、模块和约束几个层面说明了系统需求。可以看出，Mini-Chat-Bar 的核心并不只是“把消息发出去”，而是让技术讨论中的消息、结论和辅助回答能够被组织、回看和继续利用。后续章节将在此基础上进一步展开系统总体设计与具体实现。

4 系统设计

4.1 系统总体设计

本章不再重复第 3 章中的角色和场景，而是把 Mini-Chat-Bar 拆成可以直接落地的设计方案。这个系统同时要处理三类节奏不同的业务：登录后的身份初始化，一对一和聊天室消息的持续收发，以及 @AI 提问带来的历史检索和流式返回。设计时先确定的不是界面样式，而是三件事不能互相打架：消息必须写得进库，在线成员要及时收到变化，历史记录还要能回读。后面的架构、模块、界面和数据库设计都围绕这条主线展开。

4.1.1 系统架构设计

系统最终采用前后端分离架构。前端用 Vue 3 组织页面和状态^[2]，Web 端与 Electron 桌面端共用主要业务逻辑；后端运行在 Node.js 环境中，通过 Express 提供 REST 接口^[2]，再由 Socket.IO 负责在线状态和消息广播^[2]；数据层使用 MongoDB 保存用户、房间、消息、收藏和 AI 相关数据^[2]。这套组合不是按流行框架拼出来的，而是因为聊天系统里既有短请求，也有长连接，还有需要异步返回的 AI 请求，三类链路放在同一套 JavaScript 技术栈里更容易联调。

如果按一次完整操作去看，这套架构的调用顺序比较清楚。浏览器或 Electron 客户端先发起登录和资料请求；进入具体会话后，前端再通过消息读写、房间交互和收藏相关入口完成日常操作；需要让其他在线成员立刻看到的变化，则交给私聊提醒、房间广播和在线人数更新等实时事件；AI 问答与讨论总结另外走独立的智能服务入口。整体关系如图 ?? 所示。

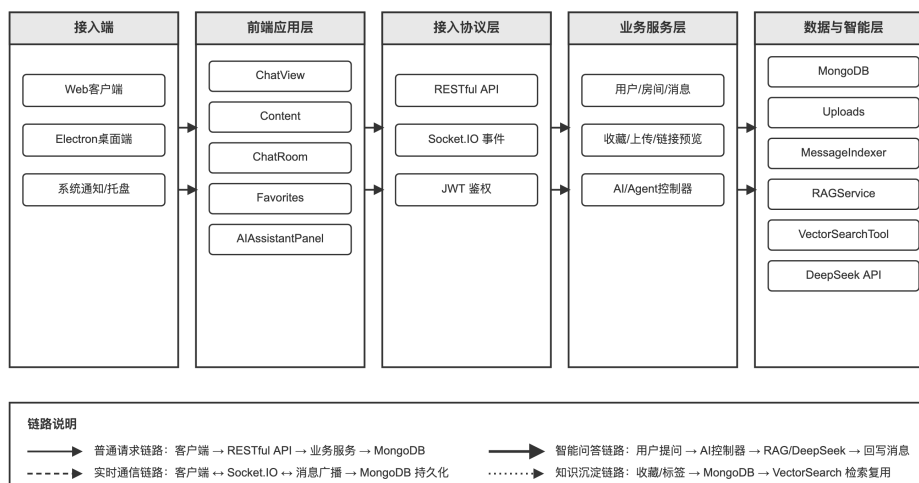


图 4.1 系统架构图

这样分层以后，登录失败、消息落库异常和 AI 超时不会混在一起。前端主要处理界面、交互和本地状态；后端统一负责校验、写库和广播；模型服务只在需要时被调用，不会把最基础的聊天链路拖进去。后面如果要补充新消息类型、替换模型接口，或者继续扩展桌面端能力，也能控制在相对清楚的边界里。

4.1.2 系统总体流程设计

把用户从进入系统到结束讨论的过程顺着写开，大致会经过下面几步：首先，用户通过邮箱密码、邮箱验证码或第三方授权登录，服务端返回 token 与基础资料；其次，前端保存本地会话并建立 Socket 连接，同时补齐当前用户信息；再次，用户进入私聊或聊天室后，消息先完成写库，再决定是否广播给在线成员；如果消息需要长期保留，就继续进入收藏或总结链路。这样写看起来比“登录认证、消息交互、知识沉淀”几个抽象词更笨一些，但更接近系统真实的工作顺序。

私聊和聊天室在流程上有明显区别。私聊消息先写入数据库，写入成功后再通知接收方；聊天室消息则先落到房间消息集合中，随后广播给房间成员。若用户在聊天室中输入 @AI，请求也不会直接丢给模型，而是先进入流式问答入口。后端结合历史消息检索结果组织上下文，再通过分块返回和实时广播持续回传内容。讨论结束后，用户还可以把关键消息加入收藏，或者生成阶段性总结。总体流程如图 ?? 所示。

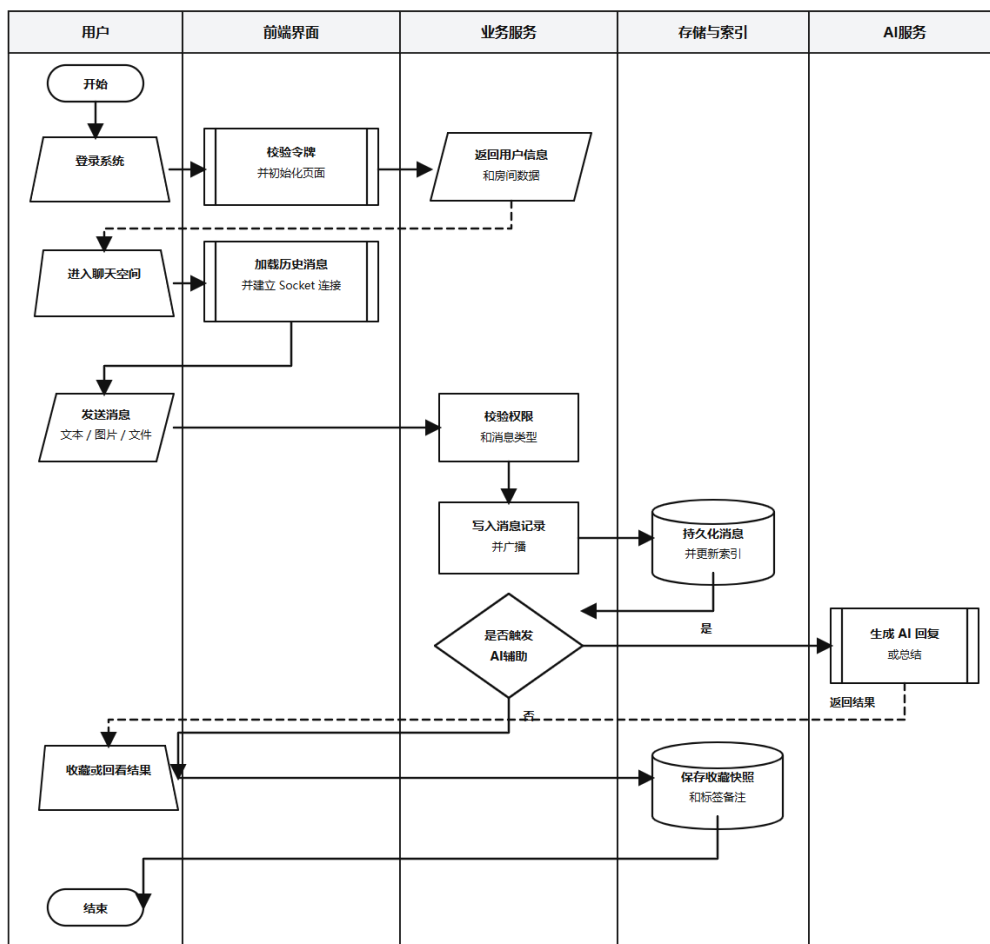


图 4.2 系统总体流程图

4.1.3 系统功能结构设计

系统功能没有按“前端一块、后端一块、数据库一块”去划分，而是按用户在界面里真正会接触到的对象拆成五块：用户管理、即时通讯、聊天室、AI助手和收藏。用户管理负责登录、身份校验和基础资料维护；即时通讯负责私聊消息、最近会话和未读状态；聊天室负责多人讨论、在线人数和房间配置；AI助手负责单人问答、聊天室@AI和讨论总结；收藏负责把聊天中的临时信息变成可回看的记录。对应结构如图 ?? 所示。

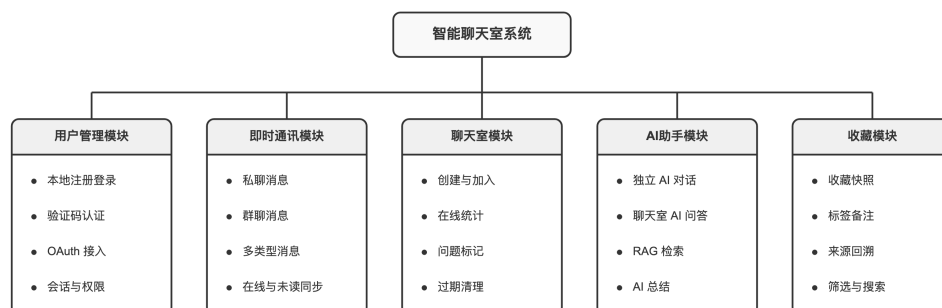


图 4.3 系统功能结构图

这五个模块不是并列摆在图上的。用户管理提供统一的 `uID`、`uName` 和 `uAvatar`，即时通讯和聊天室围绕这套身份信息组织消息，AI 模块再读取已有消息做检索和生成，收藏模块负责把其中一部分内容单独留下来。这样整理以后，系统的高频主链路就比较明确：用户先进入会话，再发生交流，最后决定哪些内容需要同步、哪些内容需要沉淀。

4.2 功能模块设计

本节不再只写“模块负责什么”，而是结合调用入口、实时事件和数据对象说明各模块的边界。

4.2.1 用户管理模块

用户管理模块看起来最普通，但它决定了后面的消息、收藏和 AI 对话到底按谁来归属。当前设计把邮箱密码登录、邮箱验证码登录和第三方 OAuth 登录都收口到同一套认证流程中。无论用户选择哪种登录方式，后端最终都会返回 `token`、用户标识和基础资料，前端再用这组信息初始化本地会话与 `Socket` 状态。

这一模块的调用顺序比较固定：用户提交登录信息，服务端完成校验并签发 JWT；前端保存 `token` 后再读取当前资料；随后建立 `Socket` 连接，把当前在线状态带到聊天列表、联系人列表和聊天室成员列表中。之所以统一成这条链路，是因为私聊、聊天室和收藏模块都依赖同一份用户身份数据，这种统一认证入口再分发业务权限的做法也符合 OAuth 安全最佳实践的约束思路^[7]。用户管理模块流程如图 ?? 所示。

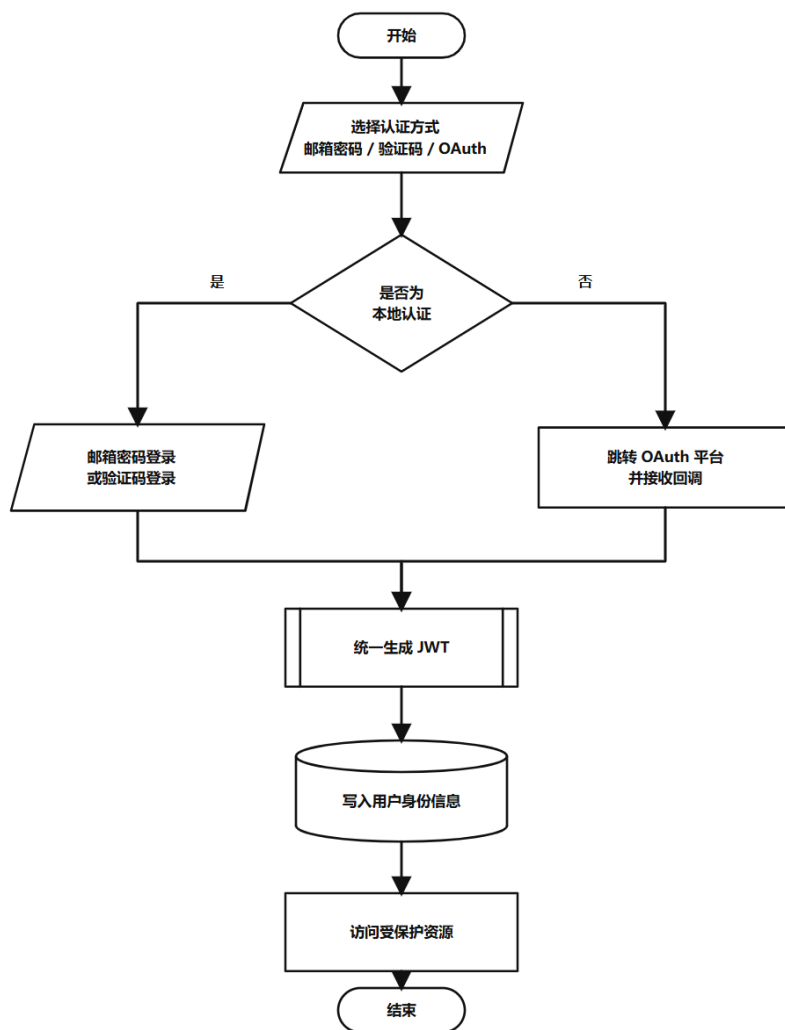


图 4.4 用户模块流程图

4.2.2 即时通讯模块

即时通讯模块承担私聊消息收发、历史回读、最近会话维护和未读状态同步。设计时把私聊链路定成“先落库，再提醒”：发送端先通过私聊消息入口写入数据库，服务端返回成功后再通知对方；最近会话列表则配合最近消息和未读统计入口维护最后一条消息与未读数。这样做的原因很直接，消息写入和列表查询需要稳定返回，而即时提醒更适合走长连接。

私聊消息的设计字段围绕 from、to、time、content、fileInfo 和 isRead 展开，既覆盖普通文本，也兼顾代码片段、图片和文件等类型。这样处理后，对方在线时能立刻收到更新，不在线时消息也已经落库，刷新页面后仍能按历史记录重新读出。即时通讯模块流程如图 ?? 所示。

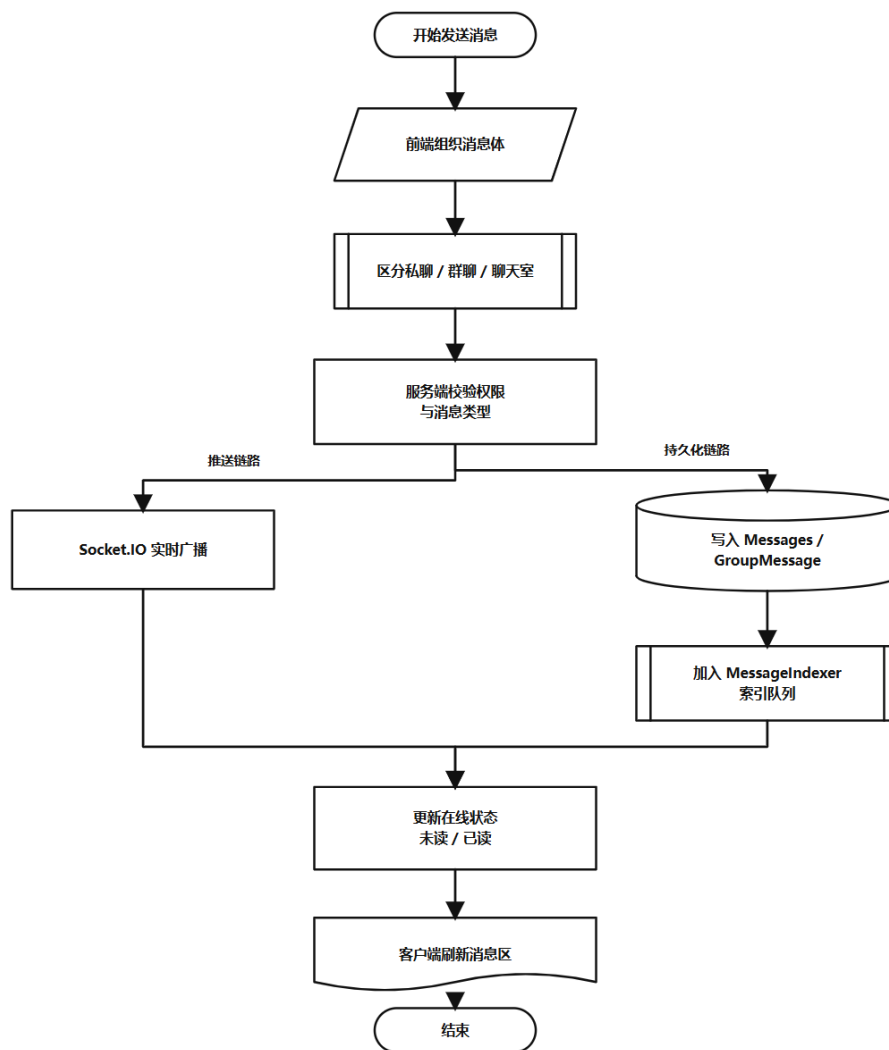


图 4.5 即时通讯流程图

4.2.3 聊天室模块

聊天室模块面向多人集中讨论的场景，设计重点不只是“多人发消息”，还包括房间如何创建、谁能加入、在线人数如何统计以及讨论结束后如何继续回看。房间对象除了 RoomID 和 RoomName 外，还要保存 Creator、Admins、Members、techDirection、joinType、inviteCode 和 expiresAt 等信息。也就是说，这里的聊天室不是普通群聊换个名字，而是带有技术方向、准入方式和有效期设置的讨论空间。

聊天室的调用顺序也比较固定。成员进入房间时，前端先完成用户与房间的绑定，服务端维护房间在线集合并回推在线人数；用户发送消息时，再写入房间消息集合并广播给房间成员；如果消息带有引用、问题标记或文件信息，也一并写入消息文档。这样设计以后，房间状态同步、消息展示和历史回看都能沿着同

一条数据链路展开。聊天室模块流程如图 ?? 所示。

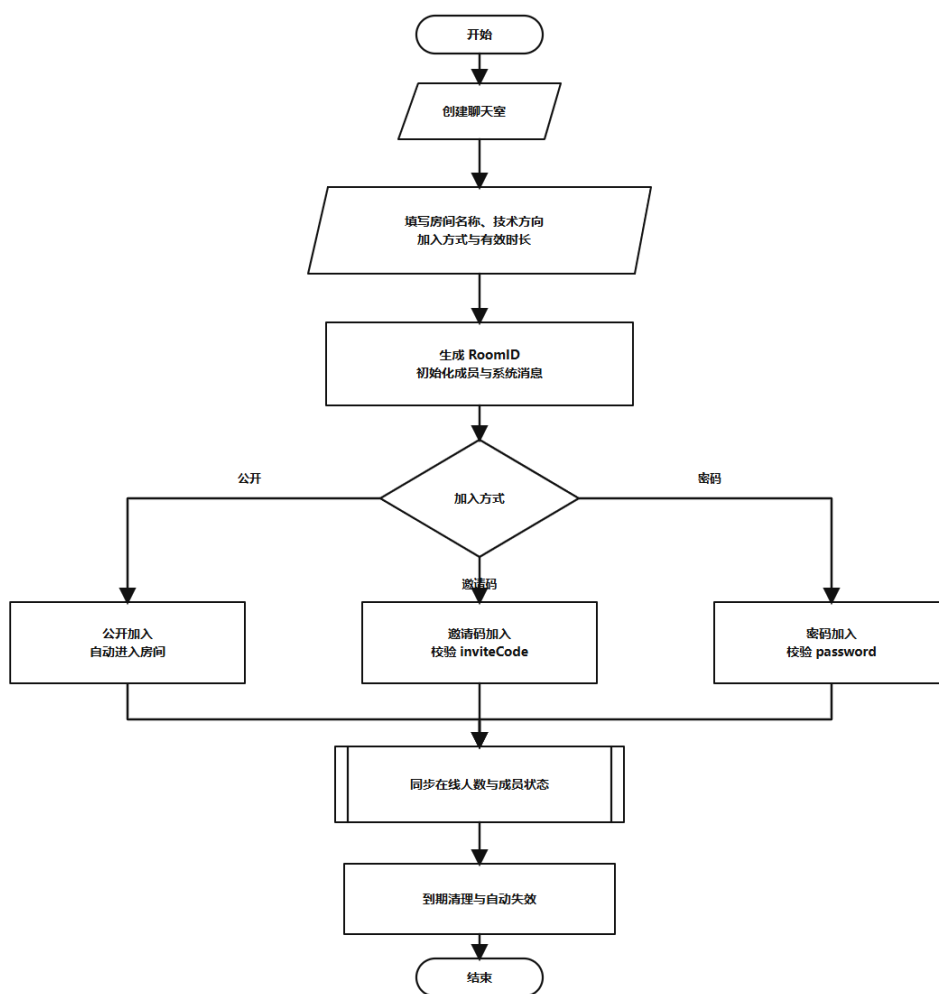


图 4.6 聊天室流程图

4.2.4 AI 助手智能问答模块

AI 助手模块分成两条线。侧边栏 AI 助手负责单人问答，适合解释代码、补充说明和快速提问；聊天室里的 @AI 负责共享回答，结果会直接回到房间消息流中，供其他成员继续阅读和引用。设计时没有把它当成一个悬在聊天之外的附加功能，而是把它并进现有消息链路里。

聊天室里的 AI 请求不会直接把一句问题丢给模型。系统先维护房间消息索引，再检索与当前问题接近的历史消息，最后把检索结果和当前提问一起送入流式问答入口。如果需要流式返回，提问者通过 HTTP 分块持续接收内容，房间其他成员则同步看到增量结果。这样处理以后，回答会尽量贴近当前房间上下文，而不是脱离前文单独生成。这样的“先检索、再补上下文、最后生成回答”的思路，与高校知识库问答系统中的 RAG 实现方式相近^[7]。在工程组织上，把检索

链路与生成链路拆开，也更接近基于 LangChain 的问答系统实现路径^[2]。企业知识库问答系统对来源内容、回答结果和业务数据分层组织的做法，也为本系统保留总结与消息快照提供了参考^[2]。由于该模块同时涉及前端页面、控制器、检索服务、模型接口和消息存储之间的顺序调用，还包含流式返回过程中的增量回写与界面更新，单纯流程图不容易把各参与对象之间的先后关系表达清楚，因此这里采用时序图来展示一次 AI 问答从发起、检索、生成到回写的完整交互链路，如图 ?? 所示。

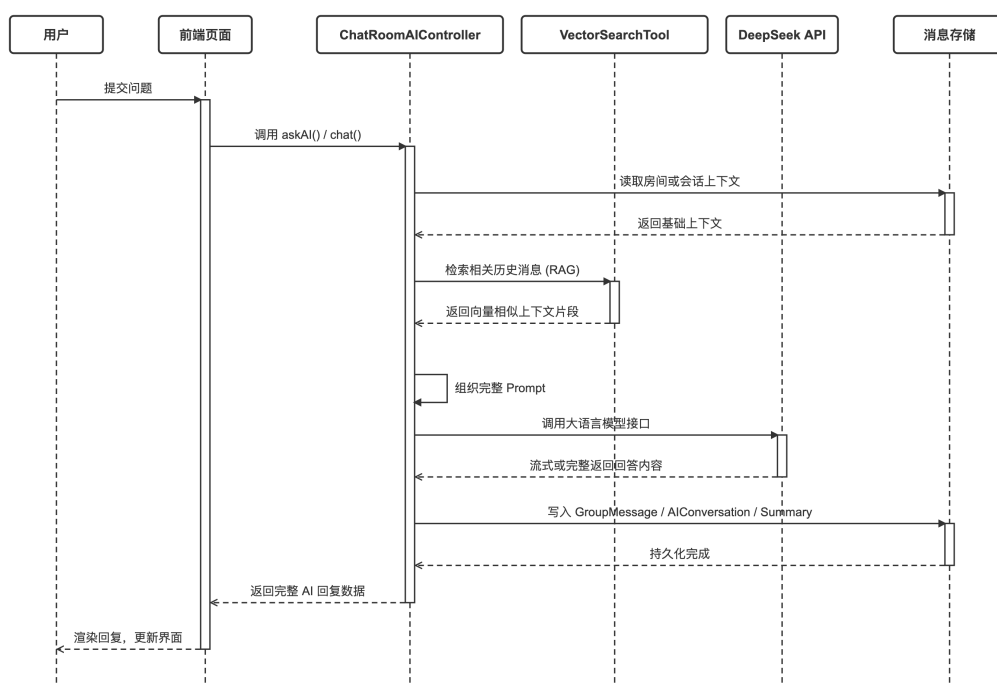


图 4.7 AI 助手智能问答模块时序图

4.2.5 收藏模块

收藏模块处理的是另一个很实际的问题：聊天里的很多内容当时有用，过几天再找就很麻烦。设计时如果只保存一条消息 ID，后续回看很容易受原始会话变化影响，所以这里没有只存引用，而是把消息快照、来源信息和用户备注一起保存下来。这样即使原始会话继续增加内容，收藏页里仍然能保留收藏当时看到的核​​心信息。

这个模块的字段设计围绕两组信息展开。一组是来源信息，例如 `userId`、`messageId` 和 `chatId`；另一组是内容信息，例如 `contentType`、`content/codeInfo`、`tags` 和 `note`。前端在消息菜单、聊天详情页或收藏页里发起操作，后端负责把快照和来源写入收藏集合。后续列表回读、取消收藏和基础筛选也都围绕这组字段展

开。收藏模块流程如图 ?? 所示。

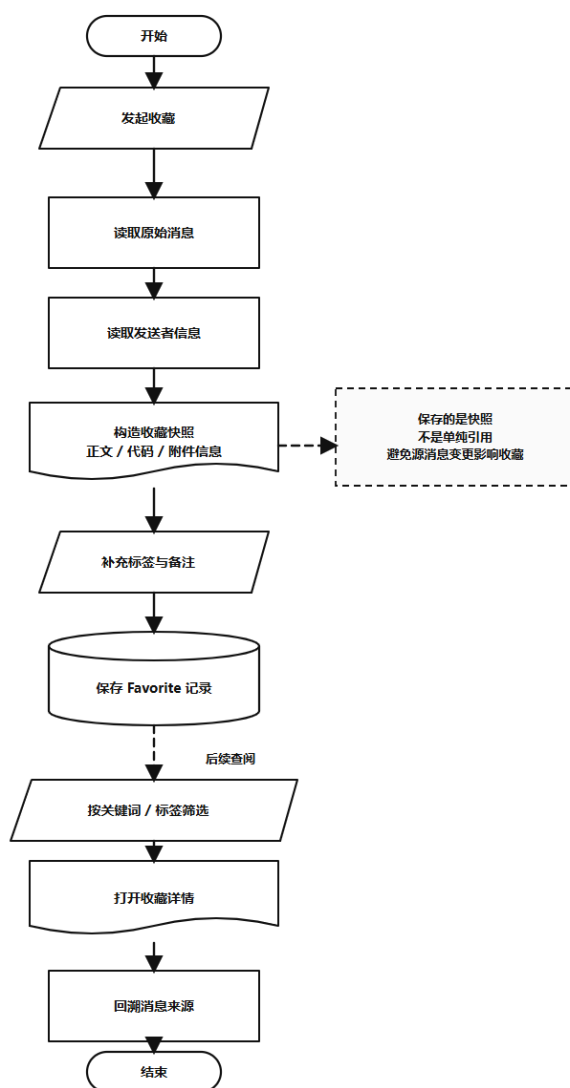


图 4.8 收藏流程图

4.3 界面设计

界面设计在这个项目里首先服务于消息流转，而不是单纯追求视觉效果。Mini-Chat-Bar 面向的是技术交流场景，页面里会同时出现长文本、代码块、文件消息、引用回复和 AI 输出，所以布局更强调列表切换效率、消息阅读效率以及高频操作是否能少走一步。

4.3.1 登录与注册界面

登录与注册界面是用户进入系统的第一步。页面采用翻转卡片形式组织登录区和注册区，同时放入邮箱密码、邮箱验证码以及第三方登录入口。这样安排的原因不是为了做动画效果，而是为了把几种认证方式放在一个页面里，减少来

回跳转。结构如图 ?? 所示。



图 4.9 登录与注册界面结构图

4.3.2 主聊天业务界面

主聊天界面统一承接系统中的高频操作。桌面端采用三栏布局，左侧负责导航和模块切换，中间栏显示聊天列表、联系人或聊天室列表，右侧栏根据当前选中的对象展示消息内容或房间详情。移动端再根据屏幕宽度切成更紧凑的视图。这里坚持三栏结构，不是为了把界面做满，而是因为会话切换、消息阅读和详情查看经常同时发生，如果每一步都跳页，技术讨论会被频繁打断。界面结构如图 ?? 所示。

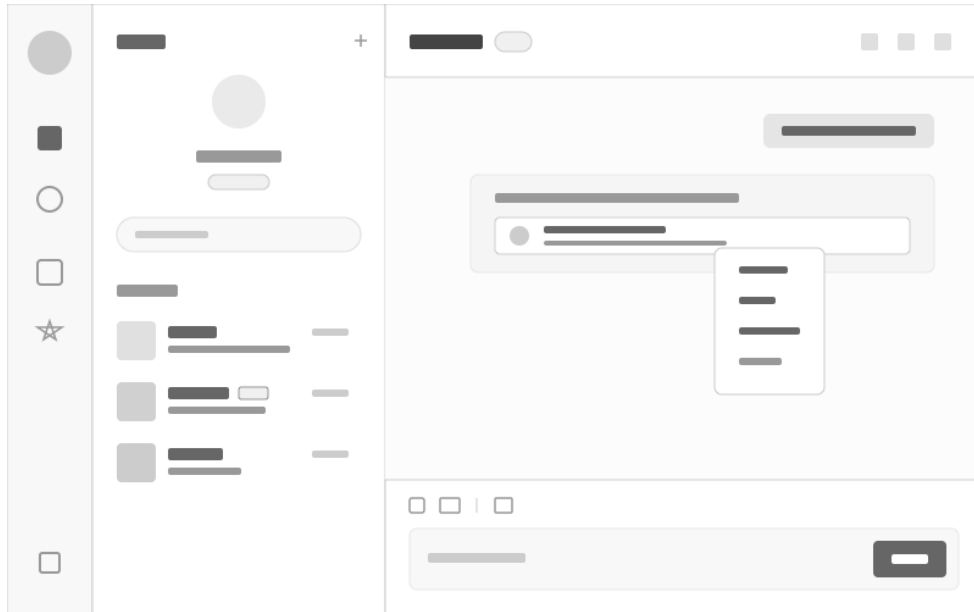


图 4.10 主聊天界面结构图

4.3.3 沉浸式技术聊天室界面

聊天室界面在普通私聊页面基础上额外突出房间信息、在线状态和协作功能。当前设计中，除消息列表外，还要同时展示房间名称、加入状态、成员在线人数、输入区以及 @AI 的返回内容。对技术讨论来说，消息区不能只显示纯文本，还要兼顾代码块、文件消息、引用回复和 AI 输出，所以这里的信息密度会比普通私聊更高。界面结构如图 ?? 所示。



图 4.11 聊天室界面结构图

4.4 数据库设计

4.4.1 E-R 图设计

虽然底层采用的是 MongoDB，但设计时仍然先按对象关系梳理实体。用户是中心对象，后面再串起聊天室、私聊消息、聊天室消息、收藏、AI 对话和聊天总结等集合。这里画 E-R 图的目的，是把“谁和谁有关”说清楚，而不是把 MongoDB 硬套成关系数据库的完全范式化设计。系统数据库 E-R 图如图 ?? 所示。

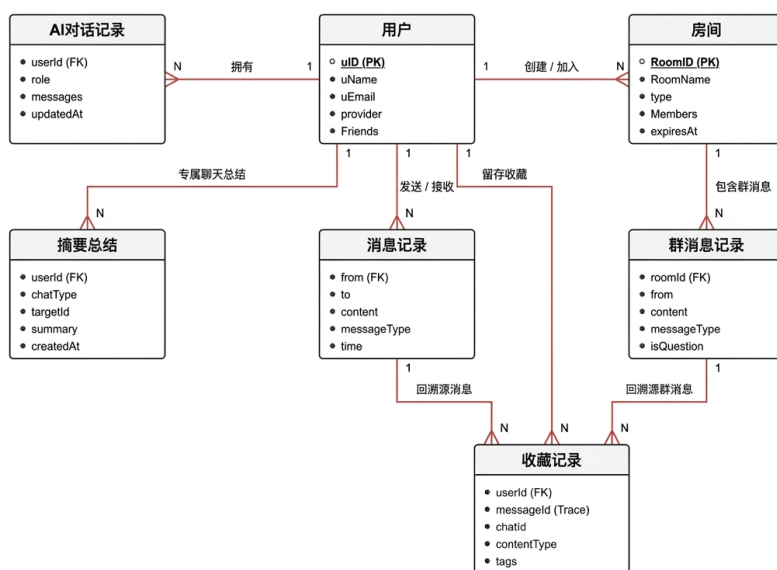


图 4.12 系统数据库 E-R 图

4.4.2 数据库逻辑结构设计

数据库结构不是先定一张总消息表，再把不同对象往里面塞字段。实际访问路径并不一样：私聊强调按双方回读，聊天室强调按 roomId 分页，收藏强调保留快照，AI 对话和聊天总结又要保存上下文。正因为读取方式不同，系统才分别设计集合和字段，而不是强行压成单一表结构。下面结合主要数据集合进行说明。

(1) 用户表设计

用户集合除了保存昵称、邮箱和头像外，还要处理邮箱登录与第三方登录并存的问题，所以表中保留了 provider、googleId 和 githubId 等字段。这些字段直接决定了登录后资料能否统一落回同一个账号，如表 ?? 所示。

表 4.1 用户表结构

字段名	主键	数据类型	是否为空	说明
_id	是	ObjectId	否	MongoDB 主键
uID	否	String	否	业务用户标识
uName	否	String	否	用户昵称
uEmail	否	String	否	登录邮箱
Password	否	String	是	邮箱密码登录时的散列密码
uAvatar	否	String	否	用户头像地址
Friends	否	Object[]	是	好友关系或联系人列表
googleId / githubId	否	String	是	第三方登录平台返回的唯一标识
provider	否	String	否	当前账号来源类型

(2) 聊天室表设计

聊天室集合保存的不是一串群聊名称，而是一组和讨论空间直接相关的配置。房间名称、创建者、管理员、成员、技术方向、加入方式和有效期都会影响后续谁能进入房间、如何统计在线人数以及房间是否需要归档，如表 ?? 所示。

表 4.2 聊天室表结构

字段名	主键	数据类型	是否为空	说明
_id	是	ObjectId	否	文档主键
RoomID/ RoomName	否	String	否	房间标识与房间名称
Creator/Admins	否	String[]	是	房间创建者和管理员列表
Members	否	Object[]	是	成员列表及加入信息
type	否	String	否	房间类型或加入方式分类
techDirection	否	String	是	房间所属技术方向标签
joinType / inviteCode	否	String	是	公开加入、密码加入等准入信息
expiresAt	否	Date	是	房间到期时间

(3) 私聊消息表设计

私聊消息集合围绕一对一聊天记录展开，字段重点放在发送方、接收方、发送时间、消息内容和已读状态上。这种结构更适合按照双方关系回读历史消息，也便于维护未读计数，如表 ?? 所示。

表 4.3 私聊消息表结构

字段名	主键	数据类型	是否为空	说明
_id	是	ObjectId	否	消息主键
from/to	否	String	否	发送方和接收方标识
time	否	Date	否	消息发送时间
content/fileInfo	否	Mixed	是	文本内容或文件相关信息
isRead/ readTime	否	Mixed	是	已读状态及已读时间

(4) 群聊与聊天室消息表设计

聊天室消息集合的字段明显比私聊复杂，因为它除了要保存 content 外，还要容纳代码片段、引用回复、问题标记、解决状态和互动反馈。技术讨论里很多信息不是一句话能讲完，所以这里的消息模型必须留出这些扩展位，如表 ?? 所示。

表 4.4 群聊与聊天室消息表结构

字段名	主键	数据类型	是否为空	说明
roomId / fromName	否	String	否	房间标识及发送者显示名
content / messageType	否	String	是	消息内容及消息类型
codeInfo/ fileInfo	否	Object	是	代码片段或文件消息的附加信息
isQuestion / isSolution	否	Boolean	否	问题标记与解决状态标记
reactions / upvotes	否	Object[]	是	表情反馈或互动数据

(5) 收藏表设计

收藏集合保存的是用户在聊天过程中挑出来的消息快照，而不是单纯记录“收藏过哪条 ID”。这样设计后，后续做分类、备注和回看会更稳定，如表 ?? 所示。

表 4.5 收藏表结构

字段名	主键	数据类型	是否为空	说明
userId / messageId	否	String	否	收藏所属用户与原始消息标识
chatId/ contentType	否	String	否	来源会话标识及内容类型
content/codeInfo	否	Mixed	是	收藏时保存的消息快照及补充信息
tags / note	否	Mixed	是	用户添加的标签与备注

(6) AI 对话表设计

AI 对话集合主要服务侧边栏 AI 助手，重点保存连续问答上下文和角色提示词。这样处理后，用户在同一轮会话里继续追问时，不需要每次都从空白状态重新开始，如表 ?? 所示。

表 4.6 AI 对话表结构

字段名	主键	数据类型	是否为空	说明
userId / role	否	String	否	所属用户与当前对话角色
rolePrompt	否	String	是	角色提示词
messages	否	Object[]	是	历史对话消息列表

(7) 聊天总结表设计

聊天总结集合保存的是私聊或聊天室中的阶段性总结结果，既要记录总结正文，也要保留消息数量和统计信息，方便后续追溯总结是基于哪一批消息生成的，如表 ?? 所示。

表 4.7 聊天总结表结构

字段名	主键	数据类型	是否为空	说明
targetId/ chatType	否	String	否	总结来源对象及会话类型
summary	否	Object	否	总结正文与结构化结果
statistics / msgCount	否	Mixed	是	总结时使用的统计信息和消息数量

4.5 核心底层机制与部署拓扑设计

4.5.1 WebSocket 即时通信协作机理

WebSocket 机制负责把已经发生的消息变化及时推送给在线用户。这里没有让 Socket.IO 单独承担全部职责，而是先由 HTTP 完成业务校验和消息落库，再把结果推给目标用户或目标房间。这样一来，消息写入失败和广播失败可以分开排查，链路更容易看清^[7]。

在私聊场景里，系统按用户标识维护连接集合，而不是把单一 socket.id 当作唯一身份，这样同一账号多端在线时仍能正常接收消息；在聊天室场景里，系统继续维护房间级在线成员集合，成员加入或退出时同步 online-count。这样既能控制广播范围，也方便后续统计房间状态^[7]。其底层时序如图 ?? 所示。

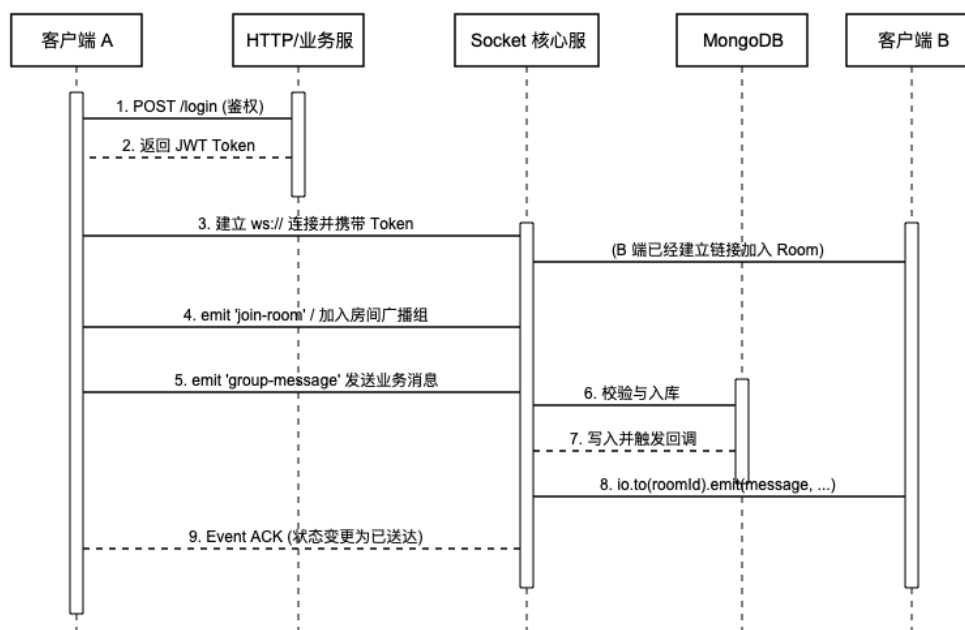


图 4.13 Socket 即时通信底层时序图

4.5.2 系统物理部署与网络拓扑架构

从部署关系看，系统由客户端、业务服务、数据库和外部模型服务四部分组成。Web 端通过浏览器访问，桌面端通过 Electron 打包程序运行；两者都通过 HTTP 和 Socket.IO 与 Node.js 服务通信。服务端内部集成 Express、Socket.IO 和数据访问逻辑，并把用户、消息、收藏和总结写入 MongoDB。只有在用户触发 AI 问答或总结时，服务端才继续调用外部模型服务，形成单独的推理链路。系统物理部署与网络拓扑关系如图 ?? 所示。考虑到本系统除了课程展示外，也希望便于个人开发者或小型团队自行搭建，部署层还预留了基于 Docker 的本地化方案。实际使用时，可将前端构建产物、Node.js 服务和 MongoDB 分别放入容器，在一台开发机或团队内网服务器上完成私有化部署。这样处理后，聊天记录、用户资料和业务配置都可以继续保留在本地环境中，迁移到新机器时也更方便统一整理。

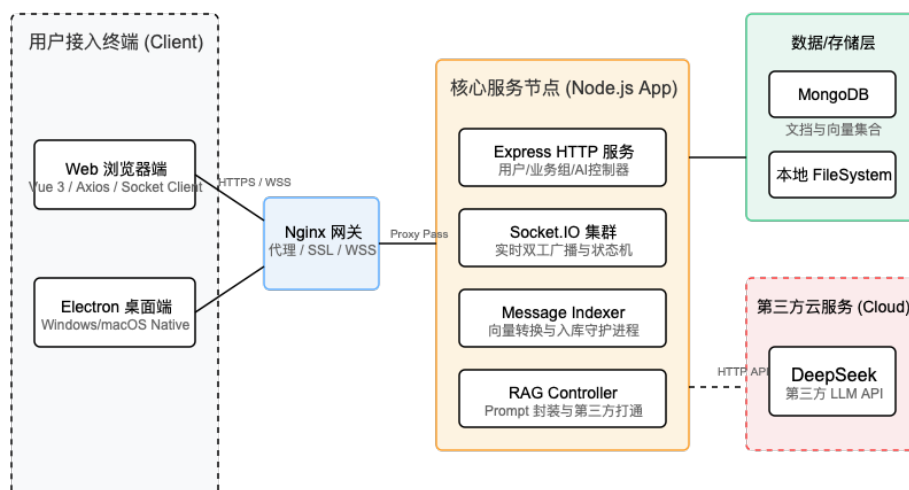


图 4.14 系统物理部署与网络拓扑架构图

4.5.3 系统核心数据流设计

除模块关系和部署关系外，还需要从数据流角度把主要链路说清楚。图 ?? 展示的是输入、处理、存储和反馈如何接起来，而不是单纯再画一张总览图。

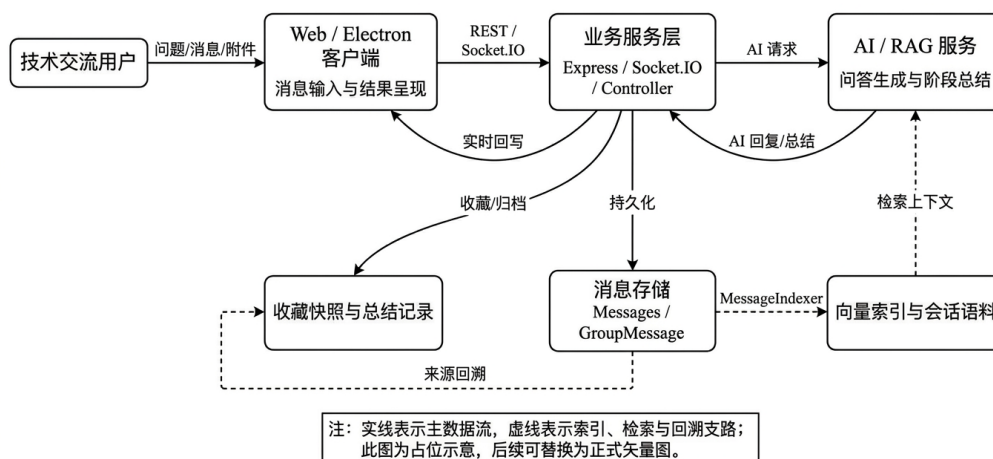


图 4.15 系统核心数据流图

从数据流上看，系统主要有三条关键路径。第一条是聊天主链路：用户在 Web 或 Electron 客户端发送消息，请求进入业务服务层后写入 MongoDB，再通过 Socket.IO 把变化同步给在线成员。第二条是 AI 辅助链路：聊天室消息进入索引后，问答请求先经过历史消息检索，再把结果与当前问题一起提交给模型服务，回答最终回写到房间消息流中。第三条是知识沉淀链路：关键消息进入收藏

集合，阶段性讨论生成总结记录，后续再通过回看和检索继续使用这些内容。和前面的架构图、流程图相比，这里更关心一条数据从输入到再次被读取，是否已经形成闭环。

5 系统实现

本章不再按源码目录逐项展开，而是顺着系统真正跑通的链路来写：先说明后端怎样完成鉴权、消息处理和 AI 问答，再说明 Web 前端与 Electron 桌面端如何把这些能力落实到界面和交互中。这样的写法少了一些泛泛的技术介绍，但更接近项目联调和功能验收时的实际顺序。

5.1 后端实现

后端部分不再单独罗列项目结构，而是直接围绕用户、聊天、AI、WebSocket 和数据模型等核心模块说明实现方式。

5.1.1 核心模块实现

(1) 用户模块

用户模块的重点不在于登录方式有几种，而在于能否把几种入口最后都收回到同一套身份体系。系统支持邮箱密码、验证码和 OAuth 登录，但无论用户从哪条入口进入，后端最后都会回到 token 签发和用户资料返回这一步，前端再据此初始化本地会话和 Socket 状态。

关键代码片段：

```
const token = jwt.sign(  
  { userId: user.uID, username: user.uName, email: user.uEmail },  
  SECRET_KEY,  
  { expiresIn: '24h' }  
)  
  
res.status(200).json({  
  message: '登录成功',  
  token,  
  user: {  
    id: user.uID,  
    name: user.uName,  
    email: user.uEmail,  
    ava: user.uAvatar  
  }  
})
```

实现逻辑说明：普通注册和登录会先完成邮箱校验与密码哈希比对，再生成 token；验证码登录多了一步验证码暂存和核验；OAuth 登录则先取回第三方资料，再按第三方 ID 或邮箱映射到本地账户。这样处理后，几类登录方式虽然入口不同，但最后都落回同一套 Users 集合，这与统一授权入口、统一令牌签发的认证实践是一致的^[2]。

资料读取接口没有被拆得很散，而是作为登录后补齐资料、设置页刷新和 OAuth 回调确认的统一入口。这样一来，用户身份链路从登录到资料刷新都比较连贯，后续聊天、收藏和 AI 模块也不需要分别适配多套身份状态。

（2）聊天模块

聊天模块是后端里最重的一部分，因为它同时要处理一对一消息、多人房间消息和几类扩展内容。这里没有把所有消息都压成一条简单文本，而是在发送时就把发送者信息、消息类型、引用内容和附件信息整理好，这样回读历史记录时不需要再做太多临时拼接。

关键代码片段：

```
const message = new GroupMessage({
  roomId,
  from: userId,
  fromName: userInfo.uName,
  fromAvatar: userInfo.uAvatar,
  content,
  messageType: messageType || 'text',
  fileInfo,
  codeInfo,
  quotedMessage,
  isQuestion: isQuestion || false
})

await message.save()
room.updatedAt = new Date()
await room.save()
io.to(roomId).emit('group-message', responseMessage)
```

实现逻辑说明：房间消息发送前会先校验房间成员身份，再补齐发送者昵称和头像并写入 GroupMessage。把 fromName 和 fromAvatar 直接冗余到消息文档

里，是为了减少历史消息回读时对用户表的额外查询；私聊部分虽然链路更短，但同样保留了发送方、接收方、消息内容与已读状态，保证列表排序、未读统计和历史检索能够对上。

聊天室里常见的引用回复、问题标记、最佳答案状态和表情反馈也没有单独做临时结构，而是直接写进消息模型。这样做的好处很直接，前端渲染消息、后端统计互动和后续 AI 检索都可以围绕同一份消息数据展开。

(3) AI 检索与流式问答实现

AI 模块没有被放在聊天链路之外另起一套系统，而是直接接在聊天室已有消息流之后。当用户在聊天室发送 @AI 问题时，后端会先整理当前提问和最近讨论，再补入检索到的历史内容，最后才进入流式推理链路，这与检索增强生成常见的“先取上下文、再生成答案”链路一致^[?]]。

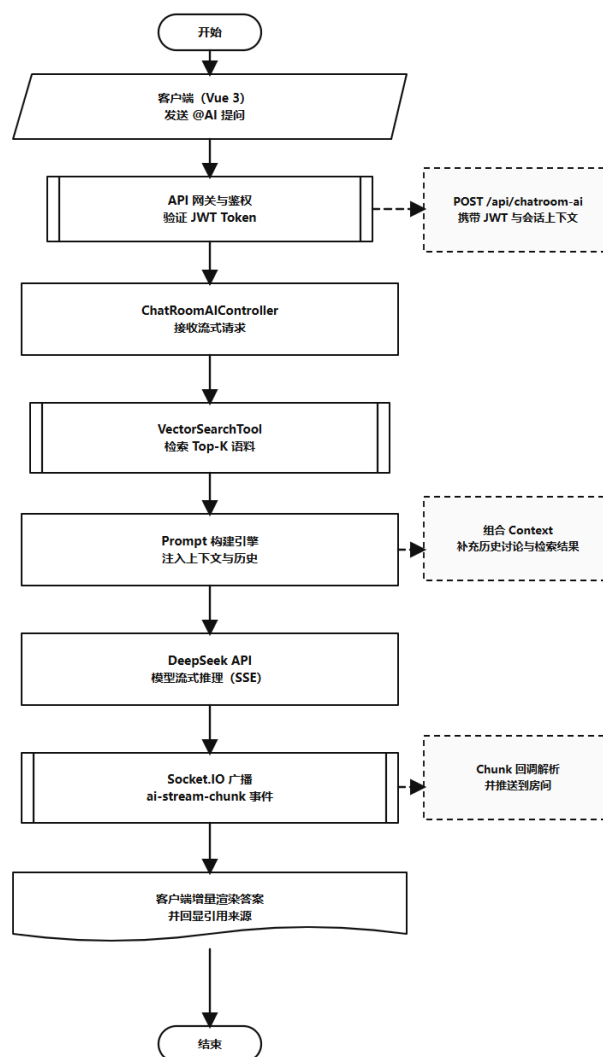


图 5.1 RAG 智能流式问答核心处理流

如图??所示, 这条链路先从房间历史中筛出与当前问题最接近的几条消息, 再把这些内容与当前问题一起组织成提示上下文, 最后发起流式请求。把历史消息索引后再回填到问答上下文中的做法, 也与面向校园知识库的 RAG 问答系统实现较为接近^[7]。

核心代码实现（截取自 ChatRoomAIController.js）:

```
// 核心逻辑: 发起网络流式请求并使用 SSE 实时回写客户端
const aiResponse = await axios.post(DEEPSEEK_API_URL, {
  model: 'deepseek-chat',
  messages: [
    { role: 'system', content: '你是一个专业的技术助手...' },
    { role: 'user', content: context } // context包含了RAG检索的 Top-K 历史
  ],
  stream: true
}, { responseType: 'stream' })

aiResponse.data.on('data', (chunk) => {
  const lines = chunk.toString().split('\n').filter(line => line.trim() !== '')
  for (const line of lines) {
    if (line.startsWith('data: ') && line.slice(6) !== '[DONE]') {
      const content = JSON.parse(line.slice(6)).choices[0]?.delta?.content || ''
      if (content) {
        fullAnswer += content
        // 1. SSE HTTP 分块下发给发起提问的客户端
        res.write(`data: ${JSON.stringify({ content })}\n\n`)
        // 2. 借助 Socket.IO 将流式片段同步广播给同一聊天室内所有其他成员
        io.to(roomId).emit('ai-stream-chunk', { roomId, content, isComplete: false })
      }
    }
  }
})
```

实现机制延伸: 该接口把 SSE 与 WebSocket 多播结合起来: 提问者通过 HTTP 分块接收回答, 房间其他成员通过 ai-stream-chunk 同步看到增量结果。这样做的收益很实际, 一是首字等待时间更短, 二是多人讨论时不会出现只有提问者先看到答案、其他成员还要额外刷新的情况。

（4）WebSocket 模块

实时通信层主要管两件事：把在线用户和连接关系维护清楚，把需要立刻同步的变化送到正确的对象手里。私聊事件和房间级事件虽然分开处理，语音交流与代码协作也拆成了独立模块，但它们遵循的是同一套实时同步思路^{[?]1}。

关键代码片段：

```
if (!users.has(userId)) {  
  users.set(userId, new Set())  
}  
users.get(userId).add(socket.id)  
  
if (!roomUsers.has(roomId)) {  
  roomUsers.set(roomId, new Set())  
}  
roomUsers.get(roomId).add(userId)  
  
io.to(roomId).emit('online-count', {  
  roomId,  
  count: roomUsers.get(roomId).size  
})
```

实现逻辑说明：这一层没有直接把 `socket.id` 当作用户标识，而是维护“用户到连接集合”和“房间到在线用户集合”两层映射。这样既能支持同一账号多端在线，也能保证房间在线人数按用户去重统计。

私聊消息、群消息、语音连麦和代码协作被拆成独立模块后，联调时定位问题也更直接。哪个链路出了问题，就回到对应事件流里检查，不会把所有实时逻辑混在一个长文件中。

（5）数据模型与辅助模块

后端除核心聊天链路外，还补充了上传、收藏、问题追踪、代码片段保存、消息索引和链接预览等辅助能力。这些内容看起来零碎，实际上都影响消息是否好发、好找和好回看。

关键代码片段：

```
const groupMessageSchema = new mongoose.Schema({  
  roomId: { type: String, required: true, index: true },  
  from: { type: String, required: true },
```

```
fromName: { type: String, required: true },
fromAvatar: { type: String },
content: { type: String },
messageType: { type: String, default: 'text' },
quotedMessage: { id: String, content: String, fromName: String },
isQuestion: { type: Boolean, default: false },
reactions: [{ emoji: String, userId: String }],
time: { type: Date, default: Date.now, index: true }
}))
```

实现逻辑说明：这些能力最终都落回到了数据结构上：引用回复、问题标记和表情反应由消息模型直接承载；收藏通过保存消息快照保证可回看；上传模块将文件地址写回消息记录；消息索引和检索服务则为后续 AI 问答提供索引基础。

（6）聊天室洞察与话题分析

聊天室在多人讨论时，不只是不断追加新消息，还需要及时看出当前讨论是否卡住、热点集中在哪些问题上。因此项目补了一层房间态势分析，用来提取未解决问题和近期热门话题。

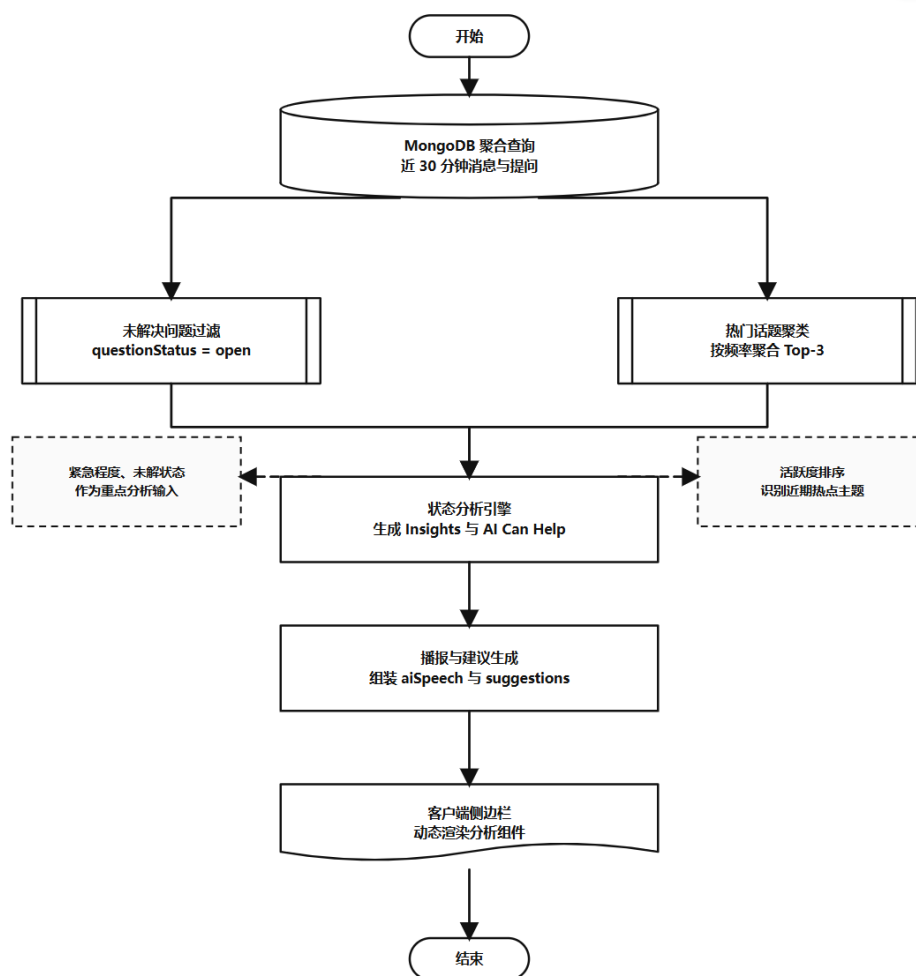


图 5.2 聊天室实时洞察与状态分析流

如图??所示，该组件主要依赖 MongoDB 聚合和内存结构完成轻量分析。

核心代码截取：

```

// 1. 查询超过30分钟处于无人问津状态的疑难技术点
const openQuestions = await GroupMessage.find({ roomId, isQuestion: true, questionStatus:
  'open' })
.sort({ time: -1 }).limit(10).lean()

// 2. 构建话题聚类探针 (Topic Map)
const topicMap = new Map()
for (const msg of recentMessages) {
  const topic = msg.content.substring(0, 20) // 对消息抽丝剥茧
  if (!topicMap.has(topic)) {
    topicMap.set(topic, { topic, messageCount: 0, participants: new Set() })
  }
}
    
```



```
topicMap.get(topic).messageCount++
topicMap.get(topic).participants.add(msg.fromName)
}
// 将 Map 对象依据消息讨论体量进行降序提取热门 Top3
const hotTopics = Array.from(topicMap.values())
.sort((a, b) => b.messageCount - a.messageCount).slice(0, 3)
```

算法层价值：系统按最近 30 分钟窗口切分消息，再利用 Map/Set 做去重聚合，以识别未解决问题和热门话题。这里没有引入复杂的离线分析流程，而是尽量在现有消息数据上完成轻量统计，最后把结果组装成 insights 报告与 aiSpeech 返回前端。

5.1.2 接口设计

结合路由、控制器和实际联调结果来看，项目常用的通信入口可以分为普通 REST、AI 和 WebSocket 三类。日常点击最频繁的是登录、资料读取、消息列表和房间消息写入这类普通 REST；智能问答和流式返回集中在 AI 接口；在线状态、群消息和增量片段则交给 WebSocket 事件。下面仅保留最能体现实现特点的几组设计。

(1) 普通 REST 接口设计。普通 REST 接口主要负责身份校验、数据持久化和列表查询，是前端大多数页面功能的基础，代表性设计如表 ?? 所示。

表 5.1 普通 REST 接口设计

接口名称	请求路径	请求方式	参数说明	返回示例
发送验证码	/api/user/ send-verification	POST	email、type	返回提示信息
邮箱密码登录	/api/user/login-email	POST	email、password	返回 token 与用户信息
获取当前用户信息	/api/user/info	GET	Bearer Token	返回 user 对象
获取私聊消息	/api/chat/ messages/:id	GET	路径参数为对方用户 ID	返回消息数组
发送群聊消息	/room/:roomId/ messages	POST	content、 messageType、 quotedMessage 等	返回消息对象

表 ?? 反映出普通 REST 更像系统的骨架，负责把身份、消息和收藏这些需要稳定回读的数据写进库里。

(2) AI 接口设计。 AI 接口主要服务于智能问答、文本解释和聊天总结等增强功能，通常需要拼接上下文并调用检索服务，代表性设计如表 ?? 所示。

表 5.2 AI 接口设计

接口名称	请求路径	请求方式	参数说明	返回示例
侧边栏智能问答	/api/agent/chat	POST	question、chatType、sessionId 等	返回答案、来源和置信度
会话总结	/api/agent/summarize	POST	chatType、 targetId/roomId、 timeRange	返回结构化总结
划词解释	/api/agent/explain	POST	text、isFollowUp	返回解释文本
聊天室流式问答	/api/chatroom-ai/ ask-stream	POST	roomId、question、 useRAG	以 SSE 分块返回结果

表 ?? 说明 AI 能力没有被压成一个大而全的入口，而是按问答、解释、总结和房

间流式协作拆开，前端可以按场景调用，后端也更容易单独调试。

(3) WebSocket 通信接口设计。 WebSocket 处理的不是所有业务，而是那些一发生就要被别人立刻看到的变化，例如上线注册、私聊通知、加入聊天室、群消息广播和 AI 流式输出同步。代表性设计如表 ?? 所示。

表 5.3 WebSocket 通信接口设计

接口名称	通信标识	触发方式	参数说明	返回示例
用户上线注册	login	client emit	userId	服务端登记在线状态并回推列表
私聊消息通知	private-message	双向	from、to、content 等	实时推送私聊消息
加入聊天室	join-group	client emit	roomId、userId	触发在线人数和成员广播
群消息广播	group-message	双向	roomId、content、fromName 等	房间成员收到完整消息对象
AI 增量输出	ai-stream-chunk	server emit	roomId、content、isComplete	前端逐段渲染 AI 回答

表 ?? 反映出实时通信层承担的是用户上线、房间状态和增量消息的即时同步。把这部分和 REST 分开以后，联调时更容易判断问题到底出在写库环节还是出在实时推送环节。

5.1.3 示例数据

下面给出一个代表性的接口返回示例，字段名与控制器实现保持一致，仅对 token 和部分 ID 做了简化。

示例 1：邮箱密码登录接口返回

```
{
  "message": "登录成功",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": "1742902338123",
    "name": "klay",
```

```
"email": "klay@example.com",  
"ava": "/images/avatar/default-avatar.webp"  
}  
}
```

5.1.4 通信机制说明

(1) HTTP 通信流程

从联调结果看，普通业务请求的基本链路比较稳定：前端通过 Axios 发起请求并附带 token，后端经鉴权中间件后进入控制器，再由 Mongoose 读写 MongoDB 并返回结果。

```
const token = localStorage.getItem('token')  
axios.get('/api/user/info', {  
  headers: { Authorization: `Bearer ${token}` }  
})  
-> auth middleware  
-> UserController.getUserInfo()  
-> Users.findOne({ uID: req.user.userId })  
-> res.json({ user })
```

(2) WebSocket 实时通信流程

实时通信与 HTTP 的分工很清楚：HTTP 负责可靠写库，Socket.IO 负责把变化立即推给在线成员。

```
socket.emit('login', userId)  
socket.emit('join-group', { roomId, userId })  
  
client POST /room/:roomId/messages  
-> GroupController.sendGroupMessage()  
-> save GroupMessage  
-> io.to(roomId).emit('group-message', responseMessage)  
-> room members refresh message list
```

(3) AI 请求调用流程

AI 请求比普通接口多了一层上下文准备与检索环节。侧边栏 AI 助手会先整理最近消息，再进入问答执行器；聊天室里的 @AI 则要拼接房间上下文、补入

检索结果后再请求流式模型接口，并把增量内容同步回房间。

```
POST /api/agent/chat
-> AgentController.chat()
-> chatAgent.execute()
  -> getRecentMessages
  -> ragService.retrieve
  -> generateAnswerWithRAG
-> return { answer, sources, confidence }
```

```
POST /api/chatroom-ai/ask-stream
-> ChatRoomAIController.askAIStream()
-> VectorSearchTool.search()
-> request DeepSeek stream
-> res.write(SSE chunk)
-> io.to(roomId).emit('ai-stream-chunk', chunk)
```

5.1.5 接口联调展示

为验证接口文档与控制器实现一致，项目在联调阶段使用 Swagger 对主要 REST 接口进行核对，避免文档里写得很完整而实际返回结构又不一致。其界面如图 ?? 所示。

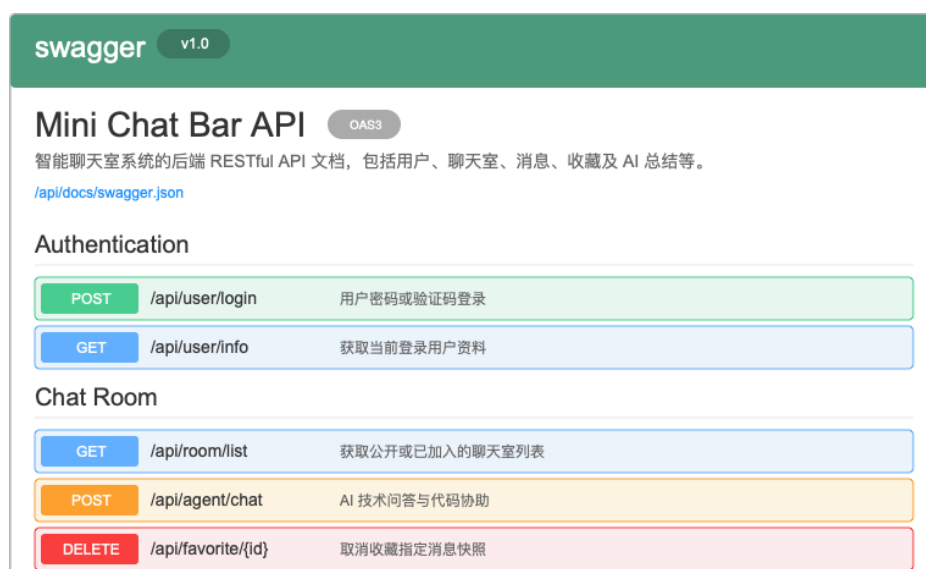


图 5.3 Swagger 接口文档

5.2 Web 前端实现

前端实现没有按技术层次拆成很多说明，而是按页面职责组织：登录入口、聊天主界面、AI 交互、消息管理和设置区域各自独立，但消息状态和会话上下文始终保持连贯。

5.2.1 登录/注册页面

登录与注册页把邮箱密码、邮箱验证码和第三方授权放在同一入口中，如图??和图??所示。这样做不是为了追求形式上的统一，而是为了让用户进入系统时少走一步，不需要在不同认证页面之间来回切换，这种按组件组织多入口认证视图的方式也更符合 Vue 3 的页面拆分思路^[2]。

交互上，页面会先完成邮箱、密码和重复密码校验，验证码登录还带有 60 秒冷却。常用接口包括邮箱密码登录、验证码登录、发送验证码、注册和第三方授权入口。用户验证通过后，前端先保存 token 和用户资料，再初始化在线状态与 Socket 连接，最后跳转到聊天主界面。流程如图??所示。

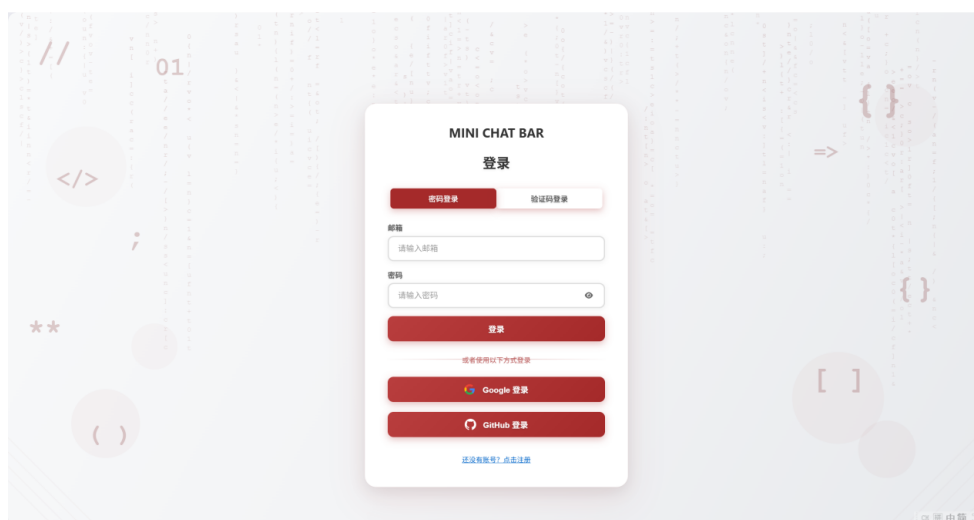


图 5.4 登录页面展示

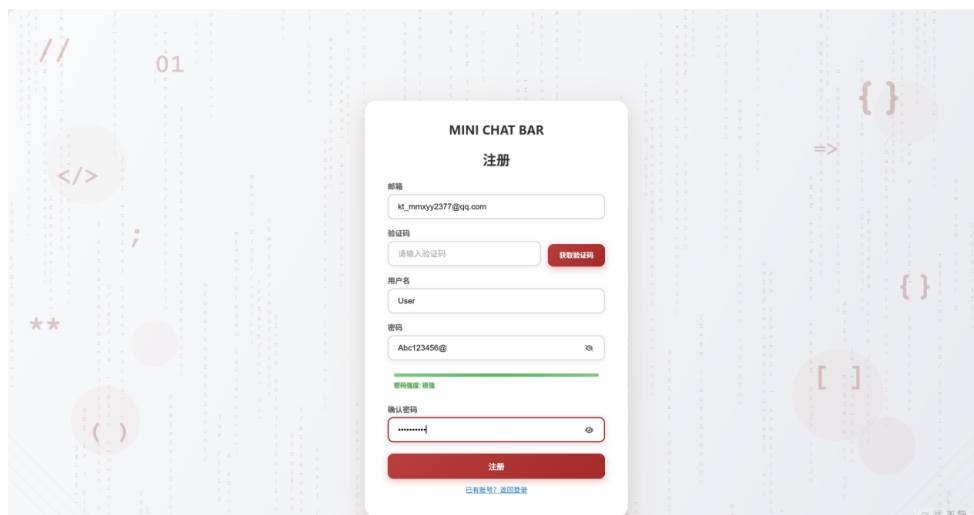


图 5.5 注册页面与表单交互展示

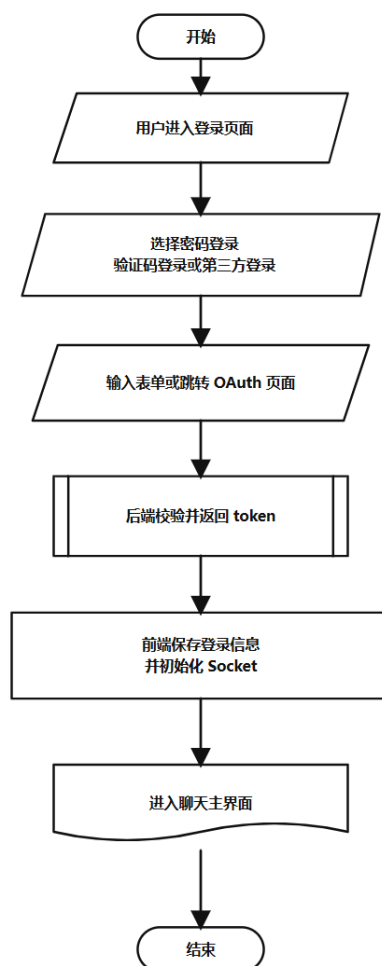


图 5.6 登录/注册操作流程图

5.2.2 聊天主界面

聊天主界面承担的是“切换对象”和“保持上下文”两件事。桌面端采用三栏布局，移动端收紧为更适合窄屏的视图，如图??至图??所示。这样安排后，导航、列表和消息详情能够同时保留在视野里，技术讨论时不必频繁跳页。

页面初始化时先检查登录状态，再根据路由决定默认展示聊天列表、联系人或聊天室列表。用户进入具体联系人或房间后，系统会把目标标识、昵称、头像和聊天类型写入会话状态，后续消息加载和状态同步都围绕这份上下文继续展开。主界面虽然不直接负责消息写入，但会频繁触发用户资料、好友列表、房间列表等接口请求。流程如图??所示。

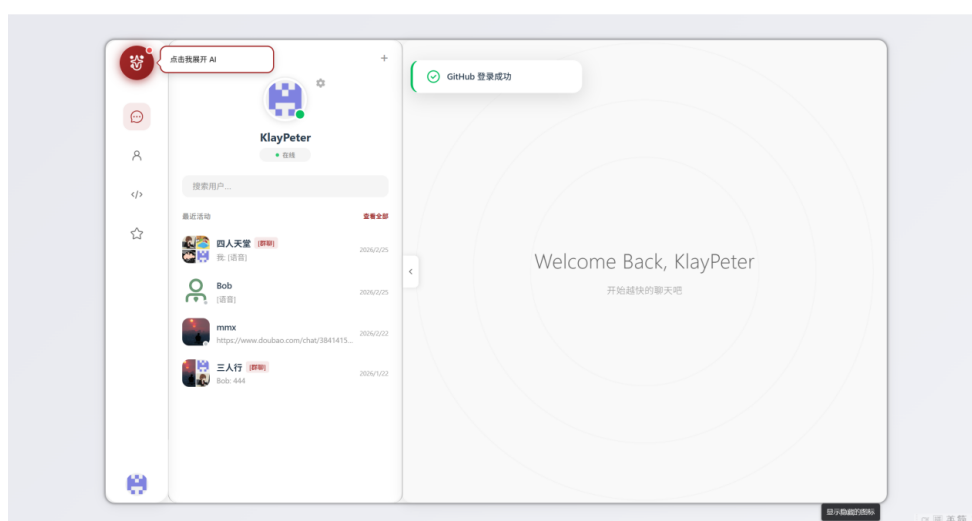


图 5.7 聊天主界面总览

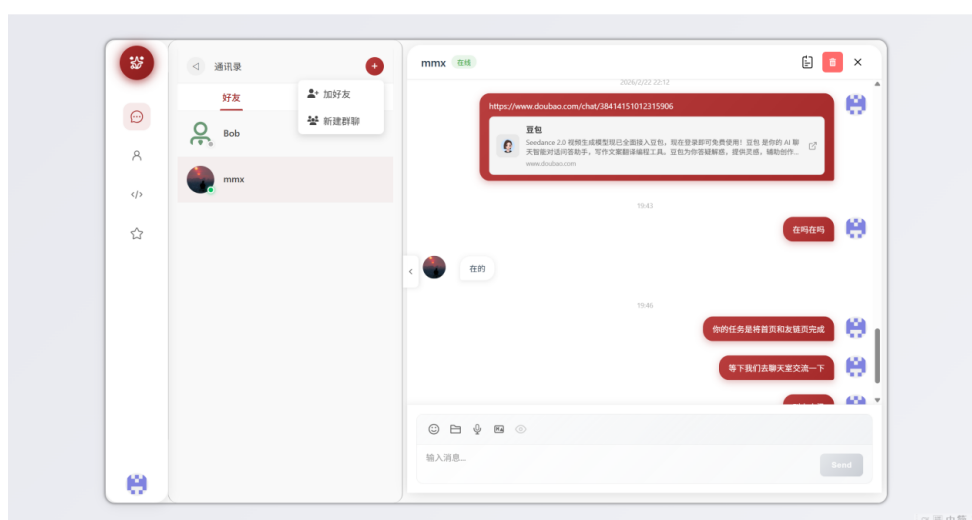


图 5.8 通讯录列表界面展示

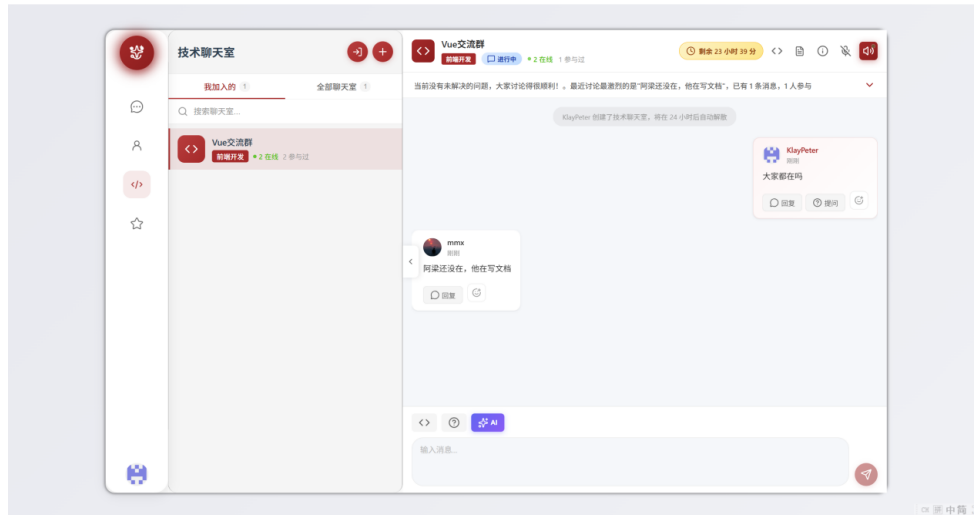


图 5.9 聊天室列表界面展示

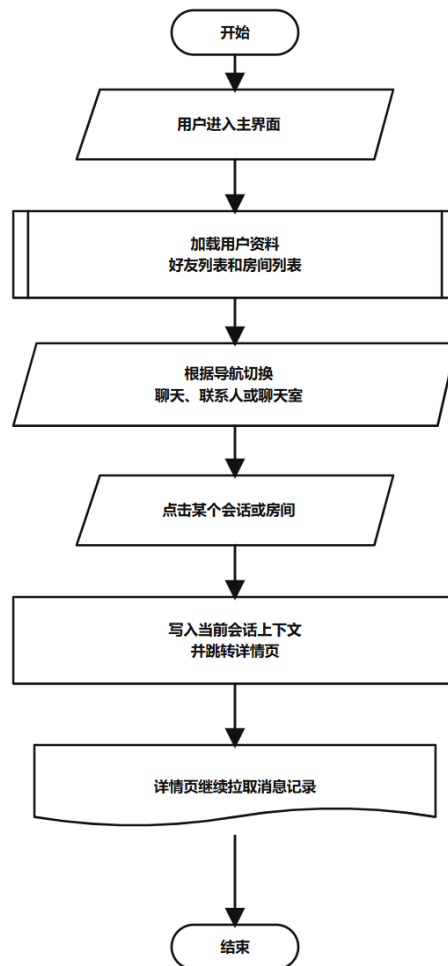


图 5.10 聊天主界面操作流程

5.2.3 AI 对话界面

AI 对话在前端里分成两种用法：一种是侧边栏 AI 助手，适合用户单独提问；另一种是聊天室里的 @AI 交互，适合多人围绕同一问题继续讨论，如图??和图??所示。前者更像补充说明，后者则直接参与到房间消息流里。

侧边栏模式下，问题会先进入本地消息数组，再调用智能问答接口获取答案；聊天室模式下，前端先把 @AI 问题发进房间，再触发流式问答入口，并在接收内容时持续刷新消息列表。由于后端会同步广播 AI 输出，房间成员看到的是同一段不断增长的回答，而不是各自独立的一份结果；这种围绕会话历史保持上下文连续性的前端交互，也与对话历史管理型问答系统的设计目标一致^[2]。流程如图??所示。

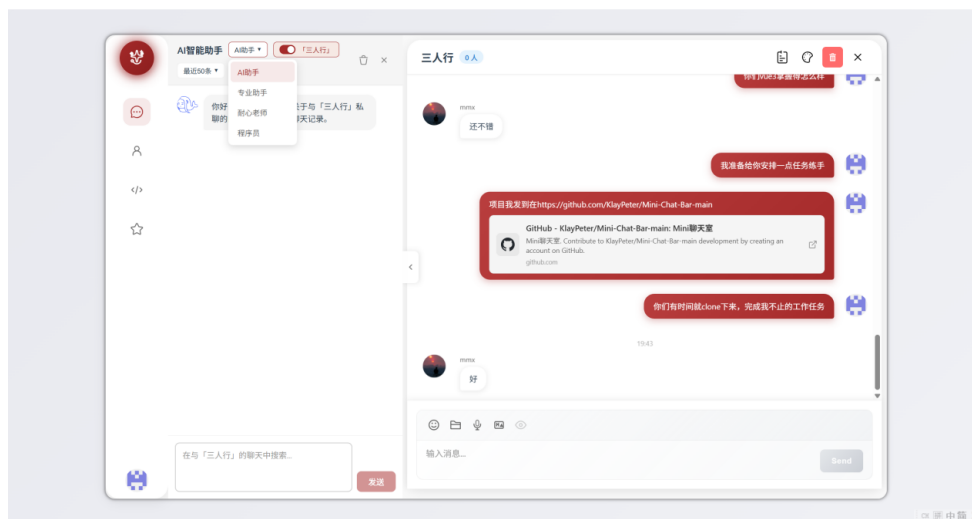


图 5.11 侧边栏 AI 助手界面

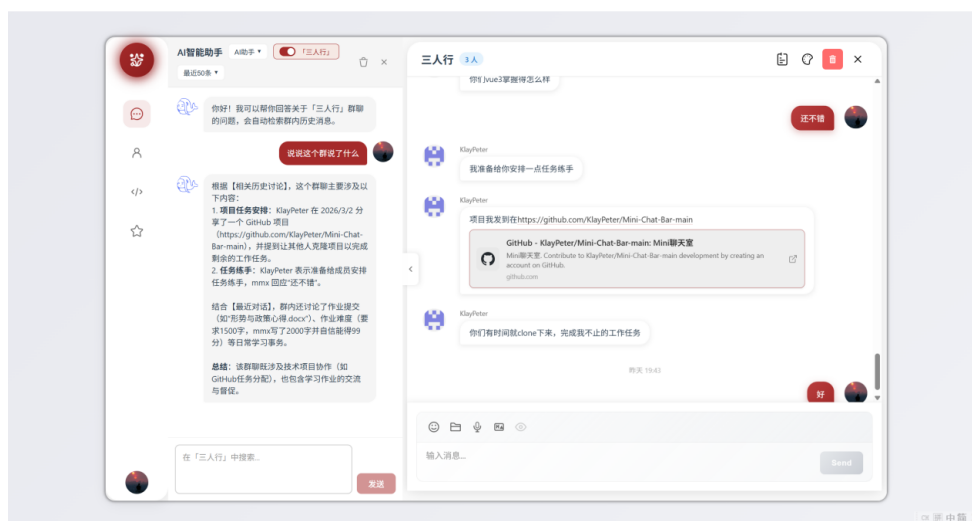


图 5.12 AI 根据历史记录检索后的回答展示

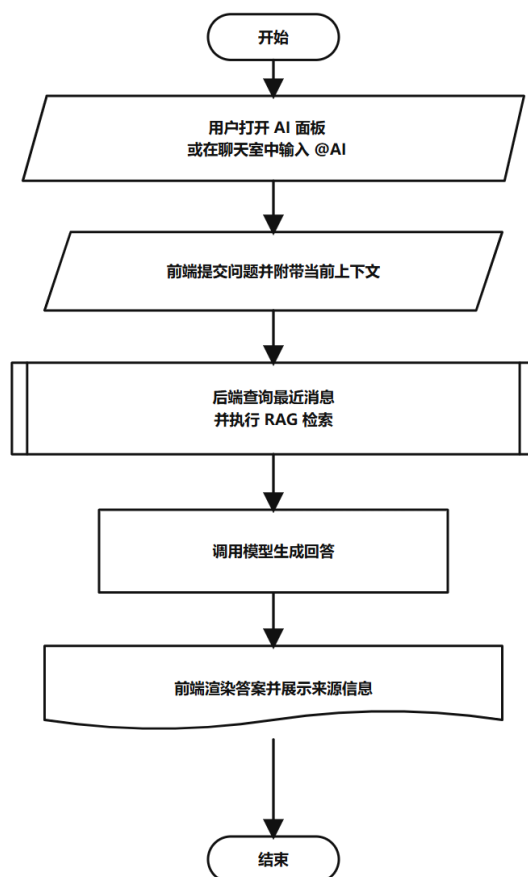


图 5.13 AI 对话操作流程

5.2.4 消息列表与会话管理

会话管理把最近会话、历史消息、搜索和收藏串成了一条连续的回看链路，如图??至图??所示。用户先看到最近会话摘要，再进入具体会话查看完整消息，需要时继续按关键词搜索，或者把关键内容单独收藏下来。

实现上，会话列表会先补齐最后一条消息和未读数并按时间排序；进入会话后再拉取完整历史记录并按消息类型渲染。这里会频繁使用最近消息、未读统计、历史消息和收藏相关接口。收藏也不是只记一个消息编号，而是会把必要快照一并保存，便于后续单独回看。流程如图??所示。

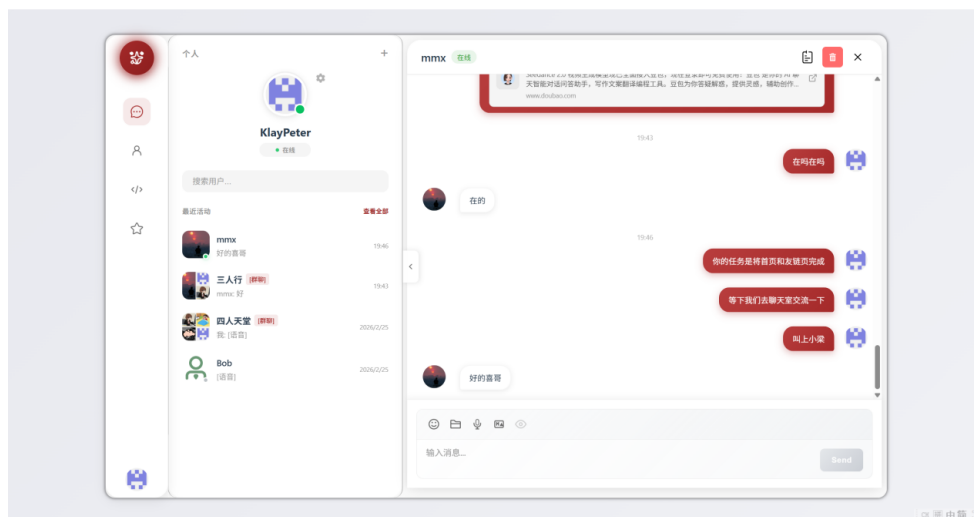


图 5.14 会话列表与私聊界面展示

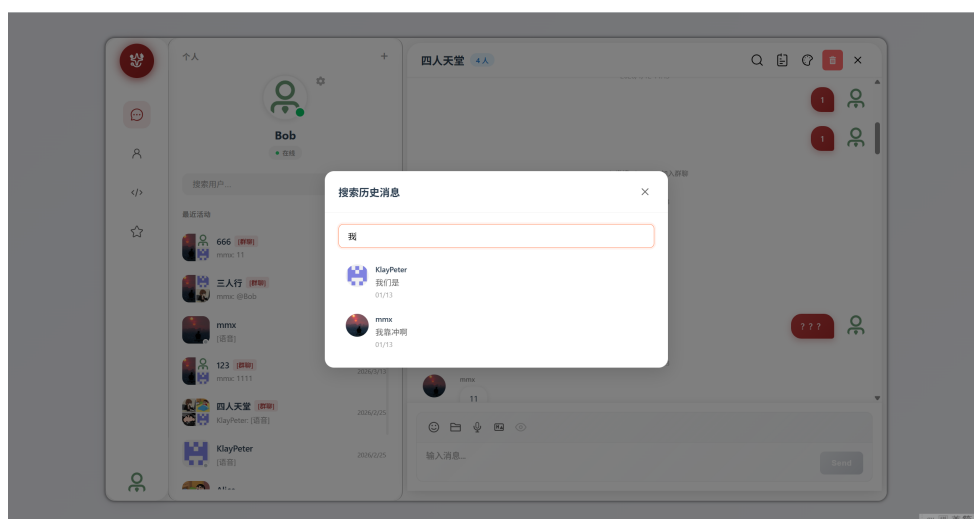


图 5.15 历史消息搜索界面展示

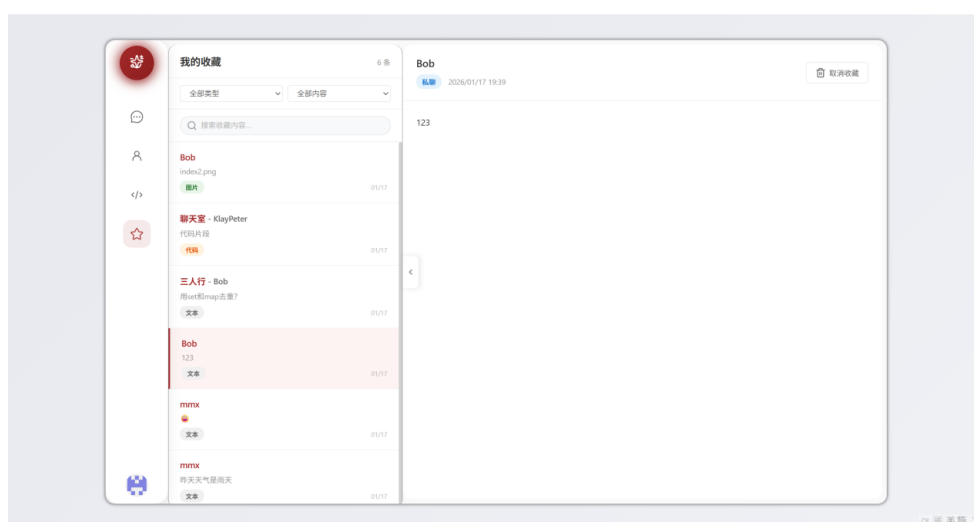


图 5.16 收藏页面展示

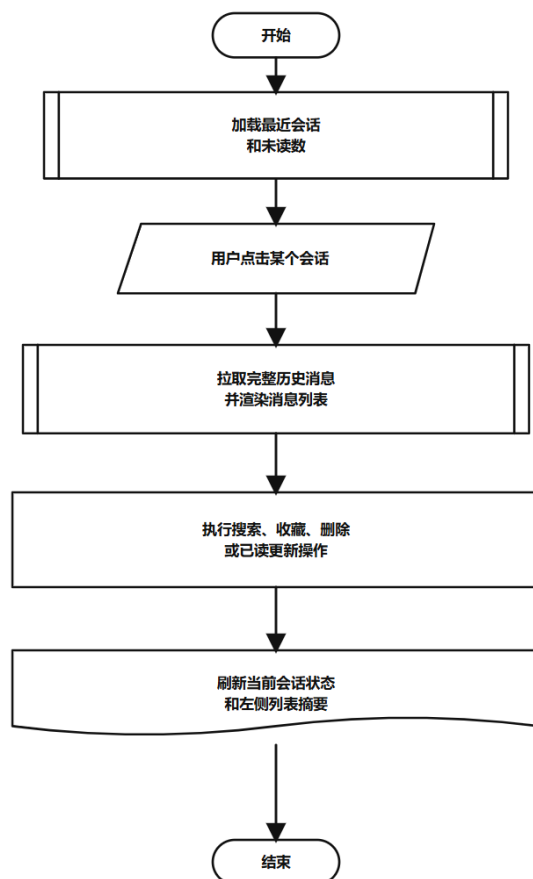


图 5.17 消息列表与会话管理操作流程

5.2.5 设置与个人中心

设置与个人中心集中处理头像、用户名、主题和退出登录，如图??所示。这个页面看起来离聊天主链路稍远，但资料一旦更新，会立刻影响最近会话列表、当前聊天页以及其他在线用户看到的显示信息。

上传头像时，前端先完成文件上传，再写回用户资料；修改用户名则直接更新用户信息。成功后本地状态会同步刷新，需要实时通知的部分再通过 Socket 广播。主题切换只保留在本地即可，因此不进入服务端链路。流程如图??所示。

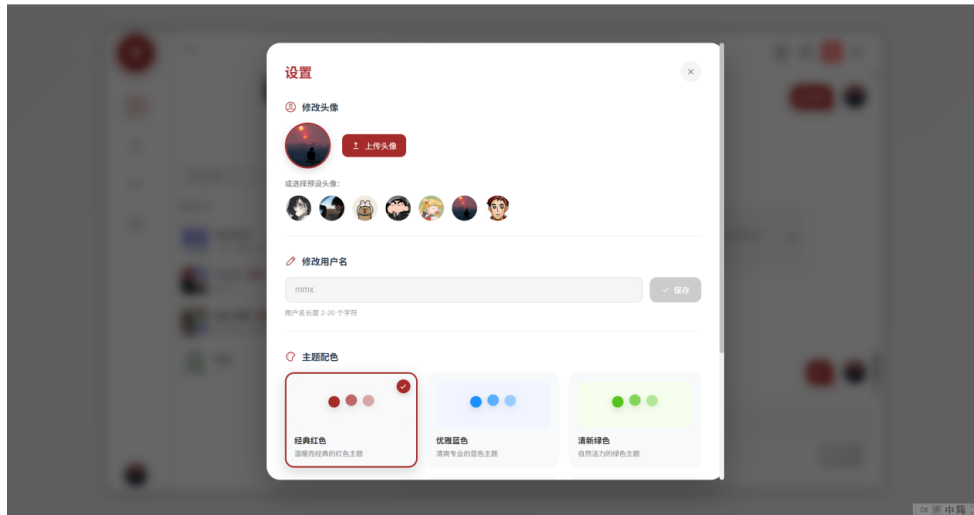


图 5.18 设置与个人中心界面展示

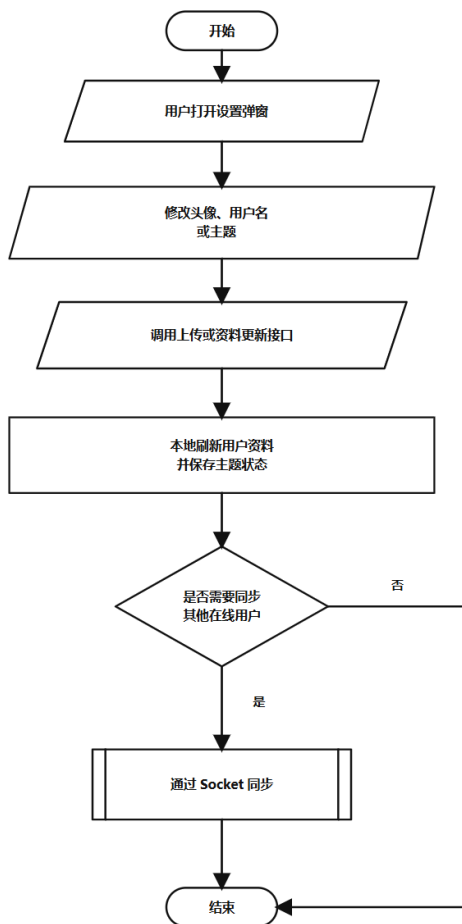


图 5.19 设置与个人中心操作流程

5.2.6 Web 端消息流转与状态管理

Web 端真正稳定运行依赖的是消息流转和状态管理。私聊发送时，前端会先完成写库，再通过 `private-message` 事件提醒对方刷新；群聊发送则先写入房间消息，再由后端广播 `group-message`。普通聊天页复用全局 `Socket` 连接，聊天室详情页再额外维护房间级在线人数和 AI 增量输出等事件。状态层只集中保存当前会话和按对象区分的消息列表，其余状态仍放在页面内部，这种拆法虽然朴素，但和当前项目体量比较匹配。

5.2.7 容器化本地部署

除开发环境直接运行外，项目也考虑了本地容器化部署。落地时可以把前端静态资源、`Node.js` 服务和 `MongoDB` 分开封装，再通过 `Docker` 在同一台开发机或团队内部服务器上启动。这样做的重点不在于把系统做成复杂运维方案，而是把部署步骤收拢到一处，便于个人开发者和小团队按需搭建，也便于把消息数据、账号信息和业务日志继续留在本地网络环境内。

5.3 桌面端实现

桌面端沿用同一套前端页面，只是在外层增加了 `Electron` 封装，因此与 Web 端的主要差异集中在窗口管理、系统托盘、本地通知和 `IPC` 通信，而不是重新实现一套独立的聊天业务。

5.3.1 系统托盘功能

桌面端最先需要处理的是窗口与托盘逻辑。程序启动后会先创建主窗口，再创建托盘图标；在 `Windows` 环境下，关闭主窗口并不会立即退出程序，而是隐藏到系统托盘继续运行。托盘左键用于显示或隐藏主窗口，右键菜单则提供“打开应用”和“退出应用”入口。

```
tray = new Tray(iconPath)

tray.on('click', () => {
  if (mainWindow.isVisible()) {
    mainWindow.hide()
  } else {
    mainWindow.show()
    mainWindow.focus()
  }
})
```

```
})
```

桌面端在收到新消息后的未读提醒和任务栏反馈效果如图??所示。也就是说，Web 端原本只有页面内提示的消息状态，在桌面环境下又补上了任务栏和托盘这两层提醒。

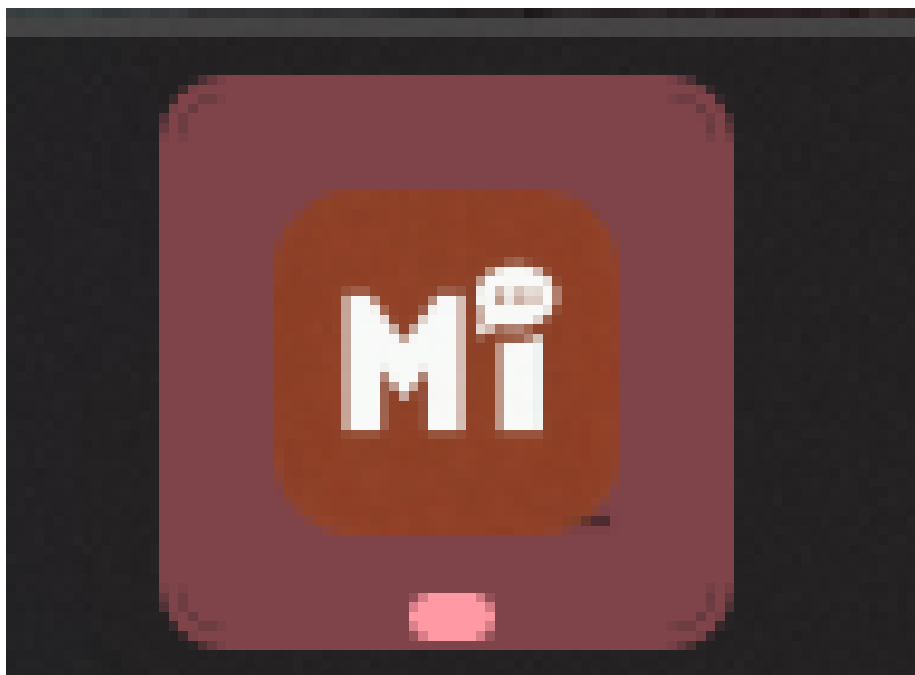


图 5.20 桌面端未读提醒展示

5.3.2 本地通知

本地通知由渲染进程和主进程配合完成。渲染进程收到新消息后，通过 `window.electronAPI.notifyNewMessage()` 把未读数发给主进程；主进程再设置任务栏闪烁、Overlay 图标、托盘提示和最近消息预览。用户点击托盘预览中的会话后，主进程会把 `open-chat` 事件发回渲染进程。

```
window.electronAPI.notifyNewMessage(unreadCount)
window.electronAPI.updateRecentMessages(recentMessages)

ipcMain.on('new-message', (event, count) => {
  startFlashing()
  setBadge(count)
})
```

系统托盘的图标、菜单入口与后台驻留形态如图??所示，使桌面端在关闭主窗口后仍能继续接收消息，并保留一个随时唤起主界面的入口。

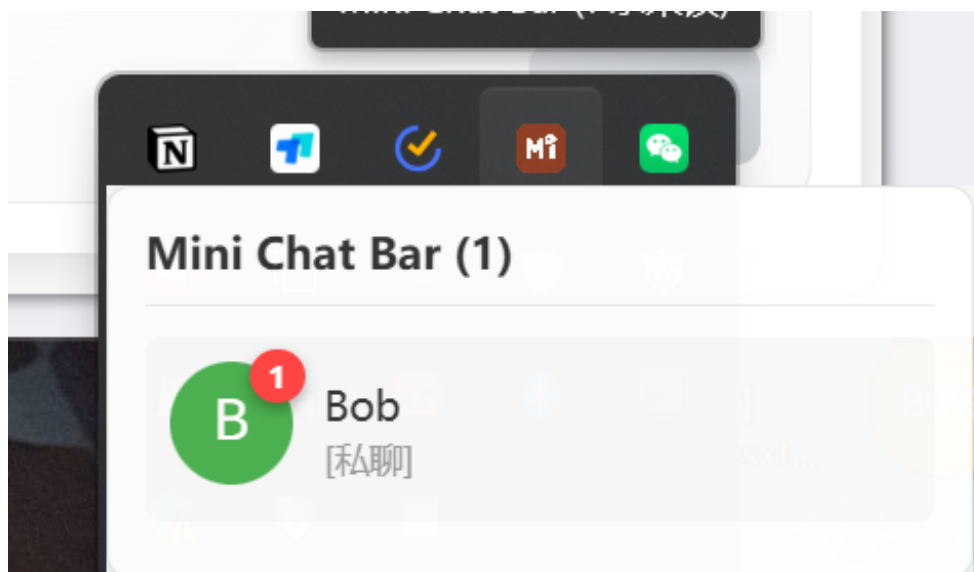


图 5.21 系统托盘展示

5.3.3 与后端和 WebSocket 通信

桌面端与后端通信的基础方式并没有变化，仍然是 Axios 调 REST 接口、Socket.IO 负责实时同步；区别在于中间多了一层 electronAPI，让渲染进程可以把新消息提醒、清空角标和打开会话等动作交给主进程处理。

因此，桌面端并不是另一套独立业务系统，而是在保留 Web 端聊天逻辑的基础上补充了桌面环境能力。核心业务代码仍只维护一份，这也是后续维护成本能够控制住的原因。

6 系统测试

本章基于当前代码、接口、脚本和打包结果，对 Mini-Chat-Bar 的主要功能进行验证。测试对象包括 Web 端、Electron 桌面端、后端接口、Socket.IO 实时链路以及 AI 问答与总结功能。测试结果一方面用于说明系统已经跑通的主链路，另一方面也用于指出当前版本仍存在的边界和问题。

6.1 测试环境与测试方法

6.1.1 测试环境配置

测试在 Windows 11 64 位单机环境完成。后端通过 `node server.js` 运行在 3000 端口，Web 端通过 Vite 运行在 `http://localhost:5173`，数据库采用本地 MongoDB，桌面端测试对象为 `ccb/dist-electron/win-unpacked/Mini Chat Bar.exe`。为减少历史数据干扰，测试账号、聊天室和消息样本均以 `ch6` 前缀重新创建，主要环境配置见表 ??。

表 6.1 测试环境配置

类别	名称	配置说明
硬件	处理器	12th Gen Intel Core i5-12450H, 12 个逻辑处理器
硬件	内存	16GB 物理内存
硬件	存储环境	本地 SSD 环境, 项目代码、数据库与打包产物均部署在同一测试主机
软件	操作系统	Windows 11 64 位, 内核版本 10.0.22631.0
软件	Node.js	Node.js 24.14.0, npm 11.9.0
软件	前端框架	Vue 3.5.13, Vite 6.3.5, Pinia 3.0.3, Vue Router 4.5.1
软件	后端框架	Express 5.1.0, Socket.IO 4.8.1, Mongoose 8.15.1
软件	数据库	MongoDB 8.0.13, 本地数据库名为 mini_chat_bar
软件	Redis 环境	Redis 5.0.14.1, 本机已安装, 用于辅助验证运行环境完整性
软件	桌面端封装	Electron 36.8.1, 打包产物为 Mini Chat Bar.exe
软件	浏览器	Chrome 146.0.7680.165
软件	辅助工具	Python 3.13.11、Postman、PowerShell、Chrome DevTools、项目内 server/test-chatroom-ai.js 与 server/test-ai-quick.js 脚本
网络与服务	服务端口	Web 端开发地址为 5173 端口, 后端接口与 Socket.IO 服务使用 3000 端口
网络与服务	向量检索模式	MessageIndexer 启动成功, 采用本地 hash embedding 模式

6.1.2 测试方法与策略

测试分为两部分进行。一部分是面向用户可见链路的黑盒测试, 主要检查登录、聊天、聊天室、收藏、AI 和桌面端提醒是否正常; 另一部分使用项目现有脚本和接口记录接口耗时、Socket 广播时延以及资源占用, 相关脚本包括 server/test-chatroom-ai.js 和 server/test-ai-quick.js。测试顺序按照“先基础链路、后集成链路、再性能验证”展开, 先以 /api/user/test 完成烟雾验证, 再依次检查认证、消息、收藏、AI 和并发场景, 这种先验证主链路、再记录接口耗时与异常结果的安排也与 RESTful 接口测试研究中的常见做法一致^[2]。

6.2 功能测试

6.2.1 功能测试用例设计原则

功能测试优先覆盖五条最常用链路：登录与会话保持、私聊与聊天室消息、AI 问答与总结、收藏管理以及桌面端提醒。每条用例都给出操作步骤、预期结果和实际结果，保证测试结论能够回溯到具体操作。

6.2.2 接口与实时通信测试

这一部分先验证后端接口和 Socket.IO 事件是否能独立工作。测试时使用 ch6userA 与 ch6userB 两个临时账号完成登录、创建技术聊天室，并通过两条 Socket 连接模拟发送端和接收端；对应测试用例如表 ?? 所示。

表 6.2 接口与实时通信测试用例

用 例 编 号	测试功能	操作步骤	预期结果	实际结果
IF-01	用户登录接口	调用登录接口提交用户名和密码	返回 200 状态码、JWT 和用户基本信息	接口正常返回 JWT，10 次重复调用平均耗时约 58ms
IF-02	用户信息接口	携带 Token 调用 /api/user/info	返回当前用户标识、昵称、头像和邮箱信息	接口 10 次重复调用平均耗时约 5ms，均能返回完整用户对象
IF-03	无效 Token 拦截	携带无效 Token 调用 /api/user/info	返回未授权结果，不泄露用户信息	接口返回未授权结果，符合权限校验预期
IF-04	聊天室消息发送接口	调用聊天室消息发送接口提交文本消息	返回消息对象，包含消息编号、发送者和时间信息	接口 15ms 内返回成功，消息被写入数据库
IF-05	聊天室消息列表读取	调用消息列表接口读取最近记录	返回消息列表，列表接口稳定返回 2 条以 上记录，时间线 与发送顺序正 确	列表接口稳定返回 2 条以 上记录，时间线 与发送顺序正 确
IF-06	私聊实时通信事件	发送端触发 private-message 事件	接收端能够实时收到完整消息内容	接收端在 6ms 内收到测试消息，内容与发送端一致

续下页

续表 6.2

用例编号	测试功能	操作步骤	预期结果	实际结果
IF-07	Socket 重连恢复	重连后再次触发 private-message 事件	重连后仍可接收实时消息，复收消息能力，且不重复投递	接收端重连后约 8ms 内恢复消息
IF-08	AI 问答接口	调用 AI 问答接口提交技术问题	返回结构化 JSON，包含回答正文和历史来源信息	单次请求耗时约 19.5s，返回约 1500 字回答；测试语料较少时历史来源列表为 0

图 ?? 汇总了这一部分的主要测试证据，包括接口返回结果、服务端终端输出、Socket 事件记录和 AI 返回界面，可直接对应表 ?? 中的测试项。

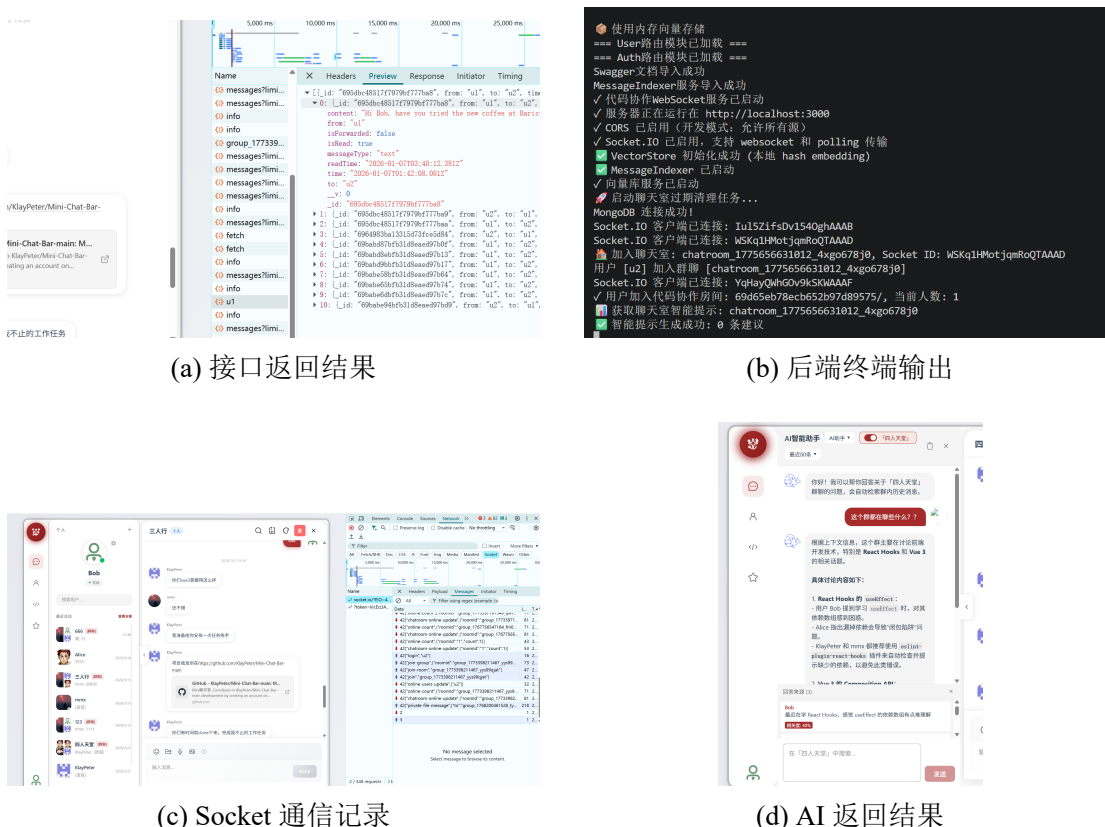


图 6.1 接口与实时通信测试证据组合图

从表 ?? 和图 ?? 可以看出，登录、资料读取、聊天室消息写入和私聊事件投递都已跑通。本地环境下 private-message 接收时延约为 6ms；AI 问答接口能够返回完整结果，但耗时明显高于普通业务接口。

6.2.3 Web 端功能测试

(1) 登录与会话保持测试

Web 端先从登录和会话保持开始验证，因为后续所有页面都依赖这条链路。测试覆盖密码登录、验证码登录、会话保持、错误凭证拦截和未登录路由保护。实测显示，登录后可从 /login 跳转至 /chats 并写入本地会话，刷新后仍能保持登录态；清空本地信息后访问受保护路由会重新跳回登录页，见表 ??。

表 6.3 Web 端登录功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-01	密码登录	在登录页输入邮箱和密码并提交	页面跳转到主聊天界面，显示当前用户信息	登录成功，主界面正常显示，接口耗时与接口测试结果一致
WEB-02	验证码登录	请求验证码后输入正确验证码并提交	登录成功并建立本地会话	验证码发送成功，60s 倒计时正常，登录后完成页面跳转
WEB-03	会话保持	刷新浏览器页面	路由守卫识别到有效 Token，不跳回登录页	刷新后仍停留在主界面，会话保持正常
WEB-04	错误凭证拦截	输入错误密码后提交表单	登录失败并给出错误提示，不进入系统主页	页面弹出密码错误提示，未发生错误跳转
WEB-05	空表单校验	不输入邮箱或密码直接点击登录	前端给出必填提示，不发起无效登录流程	页面提示“请输入邮箱和密码”，未进入成功登录流程
WEB-06	未登录路由拦截	直接访问收藏页或聊天室页等受保护路径	自动跳转回登录页并清除残留本地数据	访问受保护路由时立即跳转至 /login，残留本地数据被清理

这一部分重点记录了本地存储、页面跳转和路由守卫状态，相关证据统一汇总到图 ??。测试结果说明登录页、路由守卫和本地会话保持都已正常工作，错误凭证和空表单也会被前端拦截，没有出现异常跳转。

（2）聊天与聊天室功能测试

聊天与聊天室测试直接围绕当前前端界面展开,验证对象包括 ChatView.vue、Content.vue、ChatRoomListPanel.vue 和 ChatRoomDetail.vue。这一部分重点检查消息是否写库、历史记录是否能回读、房间是否能正常创建和加入,见表 ??。

表 6.4 Web 端聊天与聊天室功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-07	私聊消息发送	在聊天界面选择联系人并发送文本消息	对话区新增消息, 后端保存对应私聊记录	消息即时回显, 数据库中生成对应私聊记录
WEB-08	私聊历史回读	重新进入会话或刷新页面后加载消息列表	能读取历史消息并按时间顺序显示	刷新后历史消息正常加载, 最新消息位于列表末尾
WEB-09	创建公开技术聊天室	在聊天室页填写房间名称、技术方向 and 公开加入方式后提交	生成新聊天室并自动出现在房间列表中	测试房间创建成功, 列表基础回读时可见新建房间
WEB-10	加入密码聊天室	在聊天室列表中选择目标房间并输入密码加入	加入成功并看到历史消息或系统加入提示	输入正确密码后成功进入房间, 并生成加入系统消息
WEB-11	聊天室消息显示与空消息拦截	分别发送正常文本和仅空格内容	正常消息成功显示, 空消息被拦截不提交	文本消息正常显示, 空白输入不会触发发送, 消息顺序一致

测试过程中补充记录了私聊回读、公开聊天室创建、密码房间加入和空消息拦截四条关键路径, 相关证据统一汇总到图 ??。结果显示, 私聊发送、历史回读、公开房间创建和密码房间加入都能正常完成, 空白消息也会在前端被拦截, 没有进入后端链路。

（3）AI 问答与总结功能测试

AI 功能测试主要检查三条链路: 聊天室内直接提问、输入 @AI 的快捷提问, 以及讨论总结生成。为保证结果可重复, 测试前先向房间写入 4 条样本消息。实测 AI 问答约 19.5s、总结约 11.8s, 刷新后回复仍可回读, 见表 ??。

表 6.5 Web 端 AI 功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-12	AI 问答	在聊天室中提交技术问题并等待返回	页面显示思考状态，随后插入 AI 回复消息	AI 接口调用成功，约 19.5s 后返回约 1500 字回复，页面可继续交互
WEB-13	讨论总结	打开总结对话框并触发总结生成	返回结构化总结内容，能够概括主要讨论点	总结生成成功，耗时约 11.8s，基于 5 条消息生成约 600 字摘要
WEB-14	@AI 快捷提问	在输入框中插入 @AI 前缀问题并发送	自动触发 AI 问答并在消息区追加回复	AI 输入 @AI 后先写入用户问题，再生成 AI 回复，流程闭环正常
WEB-15	低语料场景问答	提交一个与当前技术方向相关的问题	仍能返回有效回答，不因历史消息较少而报错	样本消息较少时仍返回有效回答，但历史来源列表为 0
WEB-16	AI 回复持久化	刷新聊天室详情页并重新拉取消息	AI 回复仍可从历史消息中读取并显示	刷新后 AI 回复仍可从历史消息中读取，说明结果已成功落库

测试中补充记录了从提交问题到结果落库的全过程，相关证据统一汇总到图 ??。从测试结果看，AI 问答、@AI 快捷提问和总结生成都已跑通，且 AI 回复会落库，刷新后仍能回读。当前主要限制仍是耗时受外部模型服务影响，低语料场景下历史来源列表也可能为空。

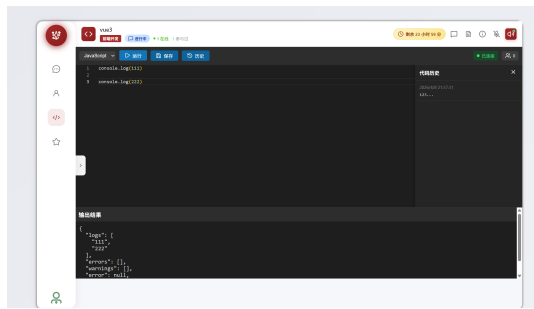
（4）收藏功能测试

收藏测试围绕添加收藏、状态同步、列表回读、取消收藏和组合筛选展开。该模块与消息列表和聊天室状态都有联动，因此除了接口返回，还需要观察前端回显是否正确。基础收藏与回读均正常，但一次带 `messageType=chatroom` 与关键字的组合查询返回了 500 状态码，见表 ??。

表 6.6 Web 端收藏功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
WEB-17	添加收藏	对目标消息执行收藏操作	收藏成功，生成消息快照和标签备注信息	收藏接口调用成功，消息被写入收藏集合
WEB-18	收藏状态同步	观察消息右键菜单或收藏状态标记	已收藏消息能够被正确识别与回显	返回消息区后星标状态正确，重新进入聊天室仍能识别已收藏消息
WEB-19	收藏列表回读	进入收藏页读取默认列表	能看到刚收藏的消息记录及来源信息	基础列表查询返回 1 条记录，能够显示测试消息
WEB-20	取消收藏	在收藏详情页点击“取消收藏”并刷新列表	收藏记录被删除，消息恢复未收藏状态	取消后收藏页记录消失，聊天室中的收藏状态同步取消
WEB-21	收藏组合筛选	在收藏页同时使用会话类型和关键字进行筛选	正常返回匹配记录	一次组合查询返回 500 状态码，说明筛选逻辑仍需进一步校验

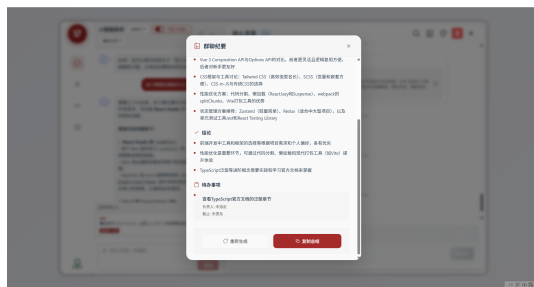
收藏测试重点记录了“收藏动作触发 - 列表回读 - 状态回显 - 取消收藏 - 组合筛选异常”这一闭环，代表性画面统一汇总到图 ??。常规路径已经可用：消息收藏后能在收藏页读到，也能正确取消。问题出现在组合筛选，带 messageType=chatroom 和关键字的查询仍然返回 500，这部分还需要单独排查。



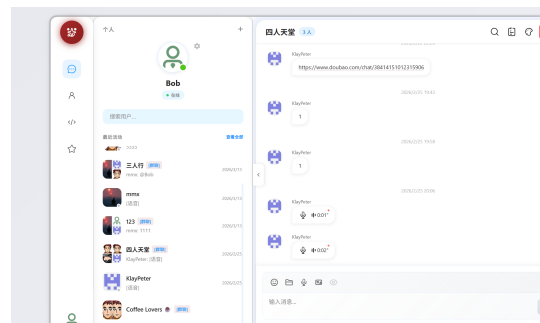
(a) 聊天室代码协作测试



(b) AI 问答结果



(c) 讨论总结结果



(d) 设置与主题切换回显

图 6.2 Web 端功能测试证据组合图

Web 端功能测试说明，登录、聊天、聊天室、AI 和收藏的主链路已经跑通，但收藏复杂筛选仍然是不稳定点。

6.2.4 桌面端功能测试

桌面端测试以 Windows 打包程序为对象，重点不是重新验证一遍所有功能，而是确认 Web 主链路迁移到 Electron 后仍可用，同时检查托盘、角标、预览和点击跳转这些桌面端特有功能，见表 ??。

表 6.7 桌面端功能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
DESK-01	桌面端启动与登录	启动 Mini Chat Bar.exe 并完成登录	主窗口正常加载，登录后进入聊天主界面	程序可正常启动，登录流程与 Web 端一致
DESK-02	托盘驻留	点击关闭按钮观察程序行为	窗口隐藏到托盘而不是直接退出程序	主窗口按设计隐藏，托盘图标仍可保留程序入口

续下页

续表 6.7

用例编号	测试功能	操作步骤	预期结果	实际结果
DESK-03	未读角标与托盘提示更新	触发新消息事件并观察任务栏角标与托盘提示变化	未读数更新，任务栏角标和托盘提示均窗口获得焦点后清空角标	随未读数变化
DESK-04	托盘悬停预览	鼠标悬停托盘图标并观察预览窗口	弹出预览窗口，悬停约 300ms 后弹出预览显示最近未读会话摘要	窗，最多展示最近 5 条未读会话
DESK-05	预览点击跳转	点击预览窗口中的某条未读消息	主窗口恢复并定位到目标会话	点击预览项后主窗口被唤起，并定位到对应会话
DESK-06	焦点恢复与角标清零	点击主窗口或让应用重新获得焦点	角标被清空，主窗口恢复焦点后角标被未读状态与界面同步	清空，未读状态与界面保持一致

桌面端部分重点记录了启动、托盘驻留、未读角标变化、悬停预览和点击跳转五个节点。图 ?? 聚焦托盘悬停预览与未读提醒效果，能够直接说明 Electron 侧消息提醒联动正常。

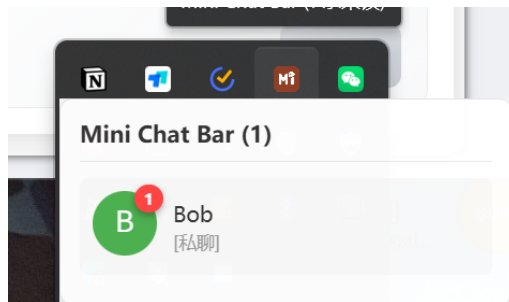


图 6.3 桌面端功能测试证据图

结果表明，桌面端已经具备可用的跨端行为：Web 端现有消息链路可以直接复用，托盘提醒和预览跳转也能正常联动。

6.2.5 功能测试结果总结

综合功能测试结果可以认为，当前版本已经跑通了登录、消息、聊天室、AI 和桌面提醒等主要演示链路。当前最明显的问题仍然是收藏组合筛选异常，以及 AI 回答质量受历史语料数量影响。

6.3 性能测试

6.3.1 响应时间测试

性能测试分为响应时间、并发广播和资源占用三部分。响应时间测试先检查日常接口的返回速度，再记录 AI 问答和聊天室总结这类耗时明显更长的调用，量化结果见表 ?? 和表 ??。

表 6.8 响应时间测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
PERF-01	登录接口响应	连续调用登录接口 10 次并记录耗时	平均响应时间稳定且无异常失败	10 次调用全部成功，平均耗时约 58ms
PERF-02	用户信息接口响应	连续访问用户信息接口 10 次并记录耗时	查询接口保持毫秒级返回	10 次调用全部成功，平均耗时约 5ms
PERF-03	聊天室列表接口响应	连续访问聊天室列表接口 10 次并记录耗时	列表查询稳定返回	10 次调用全部成功，平均耗时约 5ms
PERF-04	消息列表接口响应	连续访问消息列表接口 10 次并记录耗时	消息读取保持毫秒级返回	10 次调用全部成功，平均耗时约 6ms
PERF-05	收藏列表接口响应	连续访问收藏列表接口 10 次并记录耗时	收藏页查询稳定返回，不出现超时	10 次调用全部成功，平均耗时约 7ms
PERF-06	AI 问答响应	调用 AI 问答接口并记录返回时间	返回完整回答，不出现超时中断	单次耗时约 19.5s，返回约 1500 字内容
PERF-07	聊天总结响应	调用聊天室总结接口并记录返回时间	返回结构化摘要内容	单次耗时约 11.8s，基于 5 条消息生成约 600 字摘要

表 6.9 响应时间测试结果

测试项	平均值	最小值	最大值	结果分析
登录接口	58ms	57ms	61ms	认证、密码比对和 JWT 生成均在可接受范围内完成
用户信息接口	5ms	4ms	6ms	读取单用户资料开销较小，适合高频调用
聊天室列表接口	5ms	4ms	7ms	列表查询结构简单，返回速度稳定
聊天室消息列表接口	6.0ms	5ms	8ms	小规模消息分页读取响应平稳
收藏列表接口	7ms	6ms	9ms	收藏数量较少时查询稳定，来源名补全未带来明显额外开销
聊天室消息发送接口	15ms	—	—	消息写入数据库并广播后仍保持毫秒级返回
私聊消息发送接口	11ms	—	—	私聊接口链路较短，返回速度快
私聊 WebSocket 投递	6ms	—	—	本地实时消息转发几乎无可感知延迟
AI 问答接口	19.5s	—	—	耗时主要来自外部大模型调用，而非本地业务逻辑
聊天室总结接口	11.8s	—	—	总结耗时低于 AI 问答，适合阶段性使用而不适合高频触发

表 ?? 显示，登录、资料、列表和消息接口都保持在毫秒级；AI 问答约为 19.5s，聊天室总结约为 11.8s。当前性能瓶颈并不在本地 Node 服务，而在外部模型调用。

6.3.2 并发性能测试

并发性能测试主要观察 Socket.IO 在单机环境下的广播表现。测试脚本通过 socket.io-client 建立本地连接，统计平均时延、P95 和最大时延，结果见表 ?? 和表 ??。

表 6.10 并发性能测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
CON-01	20 客户端广播	建立 20 个 Socket 客户端并广播一条群消息	所有客户端均能收到消息，广播延迟较低	20 个客户端全部收到消息，成功率 100%
CON-02	50 客户端广播	建立 50 个 Socket 客户端并广播一条群消息	所有客户端均能收到消息，不出现掉线	50 个客户端全部收到消息，成功率 100%
CON-03	100 客户端广播	建立 100 个 Socket 客户端并广播一条群消息	广播仍能完成，延迟增长保持可控	100 个客户端全部收到消息，成功率 100%
CON-04	100 客户端连续广播	连续发送 10 轮群消息并统计投递情况	多轮广播后仍无掉线和明显积压	10 轮广播共 1000 次投递均成功，未观察到异常断连
CON-05	客户端断线重连后广播	广播新消息并观察重连客户端接收情况	重连客户端可恢复接收消息，在线客户端总数保持稳定	重连客户端可再次收到后消息，续广播，房间在线状态恢复正常

表 6.11 并发性能测试结果

并发客户端数	平均送达时延	P95 时延	最大时延	接收成功率	结果分析
20	3ms	3ms	3ms	100%	小规模广播无明显抖动
50	3ms	3ms	3ms	100%	广播链路稳定，时延未出现明显恶化
100	4ms	5ms	5ms	100%	单机环境下仍可稳定广播，时延有所上升但整体可控

如图 ??所示，并发广播测试示意图展示了单机环境下 100 客户端广播的主要观测指标。

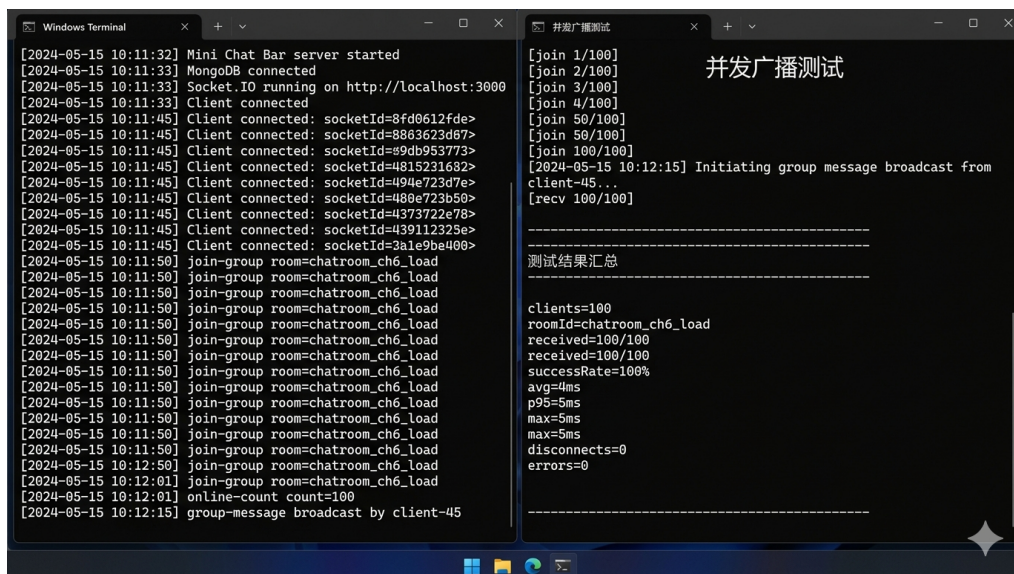


图 6.4 并发广播测试日志示意图

在 20、50 和 100 个客户端下，广播成功率均为 100%。单机 100 客户端时平均时延约为 4ms，说明当前规模下广播链路仍然稳定，但这一结果只代表本地环境。

6.3.3 资源占用测试

资源占用测试分别记录 100 客户端广播、单次 AI 问答和单次聊天室总结三种场景下的 CPU 与内存变化，结果见表 ?? 和表 ??。

表 6.12 资源占用测试用例

用例编号	测试功能	操作步骤	预期结果	实际结果
RES-01	广播场景资源占用	建立 100 个 Socket 客户端并完成一次群消息广播	CPU 和内存占用保持在可控范围内，不会出现进程异常退出	广播测试完成，进程工作集与私有内存均低于 200MB

续下页

续表 6.12

用例编号	测试功能	操作步骤	预期结果	实际结果
RES-02	AI 问答场景资源占用	对聊天室发起一次完整 AI 问答请求并记录进程占用	在等待 AI 返回期间服务进程保持稳定	AI 请求完成后进程保持正常，内存占用低于 120MB
RES-03	AI 总结场景资源占用	对聊天室发起一次完整讨论总结请求并记录进程占用	总结生成期间服务进程稳定，无明显资源抖动	总结请求完成后进程保持正常，内存占用低于 115MB

表 6.13 资源占用测试结果

测试场景	持续时间	平均 CPU 占用	工作集内存	私有内存	结果分析
100 客户端广播	1.5s	2.1%	161MB	187MB	连接数增长后内存占用上升，但仍保持在较低水平
单次 AI 问答	31.0s	低于 1%	93MB	118MB	CPU 占用不明显，说明耗时主要消耗在等待外部模型返回
单次 AI 总结	12.2s	低于 1%	90MB	114MB	与 AI 问答相比总体占用接近，但等待时间更短

如图 ??所示，资源占用监测示意图展示了广播压力和 AI 请求压力下的服务端状态差异。

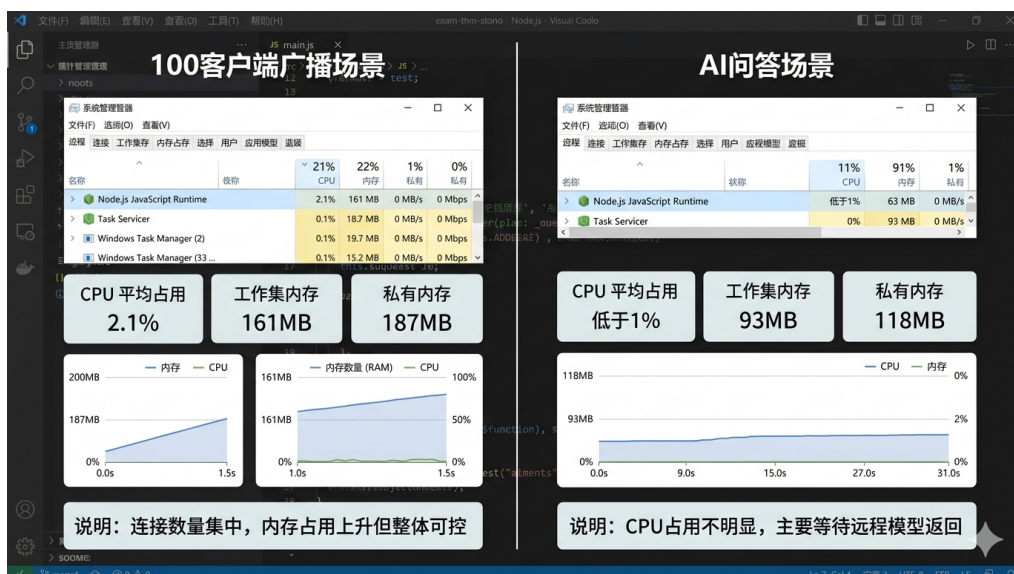


图 6.5 资源占用监测示意图

资源监测结果说明，广播场景主要抬高内存占用，AI 场景则更多消耗等待时间。本地服务端 CPU 压力不高，说明 AI 耗时主要来自等待外部模型返回。

6.3.4 性能测试结果总结

综合响应时间、并发广播和资源占用结果，系统在当前单机部署下已经能满足课程项目和答辩演示。普通接口和消息广播表现稳定；后续需要优先处理的仍是 AI 耗时和收藏复杂筛选这两个边界问题。

7 总结与展望

7.1 工作总结

到论文撰写完成时，Mini-Chat-Bar 已经完成了从需求分析、系统设计到实现与测试的主要工作。这个系统面向的不是泛社交聊天，而是技术交流场景里更常见的几条链路：用户登录后可以进入主聊天界面，在私聊和聊天室之间切换；消息支持文本、代码、图片和文件；需要反复回看的内容可以加入收藏；讨论过程中如果需要补充说明或快速整理结论，还可以调用 AI 问答和讨论总结功能。

结合整个毕业设计的推进过程，我完成的工作主要集中在几个连续阶段。前期先梳理需求、确定论文结构，并在此基础上把系统定为前后端分离架构；随后围绕用户、私聊、聊天室、收藏、AI 对话与聊天总结等模块整理业务流程和数据结构；在实现阶段继续完成 Web 端界面、Electron 桌面端适配、后端接口、Socket.IO 实时链路以及 AI 问答与总结功能的联调；最后再把需求分析、系统设计、实现过程和测试结果整理成论文。对我来说，这次毕设不是只把一个功能做出来，而是把分析、设计、编码、测试和文档整理完整地走了一遍。

从当前实现结果看，系统的主要链路已经可以闭环运行。前端已经完成登录、主聊天界面、聊天室详情、收藏页和 AI 面板等核心页面；后端已经完成用户认证、消息处理、聊天室管理、收藏管理和 AI 流式问答等服务；MongoDB 用于保存用户、房间、私聊消息、聊天室消息、收藏和 AI 相关数据。具体到业务流程上，邮箱登录后可以进入 ChatView.vue，私聊通过 `/api/chat/messages/:id` 与 `private-message` 联动，聊天室消息通过 `/room/:roomId/messages` 和 `group-message` 同步，@AI 提问则通过 `/api/chatroom-ai/ask-stream` 与 `ai-stream-chunk` 回写结果。

第六章的测试结果也说明，这套系统已经具备基本的演示和使用条件。普通接口保持毫秒级响应，私聊消息和聊天室消息可以稳定写入并同步，桌面端的托盘驻留、未读提醒和消息预览已经能够正常工作。AI 问答与总结链路虽然明显慢于普通接口，但已经跑通了从提问、检索、生成到结果落库的完整流程。对本科毕业设计而言，系统已经完成了预期的主要目标。

7.2 实践体会

这次毕业设计让我更直接地体会到，聊天系统真正难的地方往往不在某一个页面或者某一个接口本身，而在于多条链路同时工作时能不能保持一致。私聊和聊天室都不是前端把消息回显出来就算结束，还要继续确认消息有没有写入

数据库、Socket 有没有及时推送、刷新页面后历史记录能不能重新读回来。桌面端提醒也是同样的问题，弹出通知只是最表面的结果，后面还牵涉未读数、托盘状态和窗口焦点变化。如果只看单独一个点，很多问题都不算复杂；放到完整系统里之后，前后端联动和状态同步的影响就会一起出现。

另一个比较深的体会来自 AI 模块。开始做之前，很容易把它理解成“接入一个模型接口”，真正落到系统里才发现关键并不只在模型返回内容本身，而在于历史消息有没有进入索引、检索结果能不能贴近当前讨论、AI 回复生成后能不能继续以普通消息的形式保存和回看。聊天室里的 @AI 不是一个孤立入口，它必须和现有消息流共存，这一点和对话历史管理型问答系统强调的上下文连续性是一致的^[2]。这一部分做完以后，我对 AI 接入业务系统的认识比之前具体得多，也更清楚模型能力、消息数据和场景设计之间是如何互相影响的。

从个人收获来看，这次毕设最大的价值在于把一个完整项目真正从头做到尾。以往课程作业通常围绕单个功能展开，范围比较有限；这次需要同时考虑需求分析、系统设计、代码实现、测试验证和结果说明。做完整个过程后，我对接口设计、实时通信、数据库建模、前端状态管理以及问题定位都有了更直观的认识，这些经验比单独完成某一页界面更有价值。

7.3 系统不足与展望

当前版本已经能够支撑课程项目演示和小规模使用，但距离长期稳定运行的技术交流平台还有明显差距。首先，系统目前主要在本地和单机环境下开发与测试，更适合原型验证。如果后续用户数量继续增加，服务端在并发连接、消息广播和 AI 请求调度上的压力会更明显。下一步如果继续扩展，部署方式、缓存机制和服务拆分都需要重新考虑。

其次，AI 模块虽然已经跑通了问答和总结的基本链路，但它对历史语料数量和检索结果质量依赖较强。当聊天室消息较少，或者检索结果不够准确时，回答仍然会偏通用；长上下文利用效率不稳定时，这个问题还会更明显^[2]。后续更值得投入的方向，不是单纯再换一个模型，而是继续优化消息索引、检索排序和上下文拼接策略，让 AI 返回内容更贴近房间里的真实讨论^[2]。即便模型能力继续扩展，大语言模型在业务系统中的稳定性和可控性仍然需要长期处理^[2]。

再次，系统在稳定性方面还有需要补强的地方。第六章测试已经发现，收藏模块在组合筛选条件下仍会出现 500 错误。这说明主路径虽然已经跑通，但复杂路径下的异常处理还不够扎实。相比继续叠加新功能，后续更应该优先补上异常

处理、回归测试和日志分析，把现有链路先做稳。

最后，从产品方向看，当前系统已经覆盖了聊天、收藏和 AI 辅助三条主线，但对技术团队更深层次的协作支持还不够。例如，多人代码协同、任务跟踪、知识归档的结构化整理等能力，目前还没有形成完整方案。后续如果继续推进，可以在保持轻量化的前提下逐步补充这些能力，使系统从“技术讨论工具”进一步发展为“面向技术交流的协作平台”。

总体来看，Mini-Chat-Bar 已经验证了一个比较清楚的方向：技术交流系统不应只负责消息传递，还应支持历史回看、结果沉淀和适度的 AI 辅助。当前版本还算不上成熟产品，但主框架已经建立起来，后续只要继续围绕稳定性、检索质量和协作深度推进，系统仍然有较大的完善空间。

参考文献

- [1] 韩玉婷, 叶自强. 文本型聊天系统的设计与实现[J]. 计算机软件及计算机应用, 2024: 1-5.
- [2] 任海玉, 刘建平, 王健, 等. 基于大语言模型的智能问答系统研究综述[J]. 计算机工程与应用, 2025, 61(7): 1-24.
- [3] 龙举兵, 罗忠巍. HTTP 轮询、SSE 和 WebSocket 的 IoT 的 Web 管理系统的研究和应用[J]. 互联网技术, 2026(2): 1-6.
- [4] OpenAI. GPT-4 Technical Report[J]. arXiv preprint arXiv:2303.08774, 2023.
- [5] 刘雪颖, 云静, 李博, 等. 基于大型语言模型的检索增强生成综述[J]. 计算机工程与应用, 2025, 61(13): 1-25.
- [6] 周思源. 针对开发者聊天室对话的摘要方法研究[D]. 浙江工商大学, 2025.
- [7] Vue.js Team. Vue.js Guide[EB/OL]. Vue.js(2026)[2026-03-29]. <https://vuejs.org/guide/introduction.html>.
- [8] Pinia Team. Pinia Introduction[EB/OL]. Pinia(2026)[2026-03-29]. <https://pinia.vuejs.org/introduction.html>.
- [9] Express.js Team. Express 5.x API Reference[EB/OL]. Express.js(2026)[2026-03-29]. <https://expressjs.com/en/5x/api.html>.
- [10] Socket.IO Team. Socket.IO Documentation[EB/OL]. Socket.IO(2026)[2026-03-29]. <https://socket.io/docs/v4/>.
- [11] Lodderstedt T, Fett D, Campbell B, et al. The OAuth 2.0 Authorization Framework: Security Best Current Practice[EB/OL]. Internet Engineering Task Force(2025)[2026-03-29]. <https://www.rfc-editor.org/rfc/rfc9700.html>.
- [12] MongoDB, Inc. MongoDB Manual: Data Modeling Introduction[EB/OL]. MongoDB(2026)[2026-03-29]. <https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>.
- [13] 李仲岐. 基于 WebSocket 即时通信系统设计与实现[C]//第三十七届中国(天津)2023'IT、网络、信息技术、电子、仪器仪表创新学术会议论文集. 天津, 2023: 1-5.
- [14] 甘克勤, 高亮, 肖宝坤, 等. 基于检索增强生成与对话历史管理的标准智能问答系统研究[J]. 计算机应用, 2026: 1-6.
- [15] 孙业超. 基于 RESTful API 的前后端分离项目接口测试方法研究[J]. 互联网技术, 2025: 1-5.

- [16] 张龙. 基于国密 SM2/SM3 算法的保密即时通信系统设计[J]. 网络安全技术与应用, 2024 (7): 25-27.
- [17] 白晶, 张艳明, 扈韵绮, 等. 基于大语言模型和 RAG 技术的高校知识库智能问答系统构建与评价[J]. 信息技术, 2026: 1-6.
- [18] 陈文丽, 张洪波, 靳德政. 基于 LangChain 的检索增强生成知识库问答系统设计与实现[J]. 计算机应用, 2026(2): 1-6.
- [19] 陈厚创. 基于大语言模型与检索增强生成的企业知识库问答系统[J]. 计算机软件及计算机应用, 2026: 1-6.
- [20] 王彬. 基于 WebSocket 协议的网页即时通信系统研究与实现[J]. 互联网技术, 2024: 1-5.
- [21] 李华, 王磊, 张伟. 基于 WebSocket 与 Vue 的实时在线交流系统设计[J]. 电脑编程技巧与维护, 2024(5): 42-45.
- [22] Gao Y, Xiong Y, Gao X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[J]. arXiv preprint arXiv:2312.10997, 2024.
- [23] 高雅奇. 基于大语言模型和 RAG 技术的高校知识库智能问答系统构建与评价[J]. 信息技术, 2024: 1-5.
- [24] Liu N F, Lin K, Hewitt J, et al. Lost in the Middle: How Language Models Use Long Contexts [J]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173.
- [25] Zhao W X, Zhou K, Li J, et al. A Survey of Large Language Models[J]. arXiv preprint arXiv:2303.18223, 2023.

致谢

行文至此，这篇论文也接近尾声，大学四年的时光仿佛也在这一页页文字里慢慢落定。回望从选题、设计、实现到反复修改的过程，我并不是独自走到这里。一路上，老师的指引、同学的陪伴与帮助，让许多原本紧张而忙碌的时刻，都多了几分踏实与从容。

衷心感谢我的指导老师刘利副教授，在论文选题、系统设计和写作修改过程中给予我耐心而细致的指导。老师严谨认真的治学态度，也让我明白，完成毕业设计不仅是完成一项任务，更是学着把问题想清楚、把事情做扎实。

同时，也感谢身边的同学和朋友。在系统开发和论文撰写过程中，大家陪我一起讨论思路、排查问题、完善细节。许多看似平常的交流与提醒，都会在回头看时显出分量，它们像夜里一直亮着的灯，让这段毕业时光不只是匆忙，也留下了温度。

在此，向所有曾给予我帮助、鼓励与关心的老师、同学和朋友，致以真诚的谢意。那些一起思考、一起赶进度、一起把问题慢慢解决的日子，也会成为我大学生活中很珍贵的一部分。